CSE 713: Artificial Intelligence

Search Algorithms — Complete Model Answers

University of Chittagong — 7th Semester B.Sc. (Engg.) Examination

Covers Years: 2024, 2023, 2022, 2021, 2020 — Topics: UCS, BFS, DFS, Greedy, A*, IDDFS, IDA*

Contents

Heuristic Search — Theoretical Foundations

This section covers the **theory** that underpins every search-algorithm question in the exam. The year-by-year traces in later sections apply these concepts to specific graphs.

BFS (Uninformed) vs. Best-First Search (Informed)

The fundamental divide in search algorithms is between those that use **no domain knowledge** and those that use **heuristic guidance**.

Criterion	BFS (Uninformed / Blind)	Best-First Search (Informed / Heuristic)
What guides expansion?	**Structural position** — depth from the root. BFS expands level by level, regardless of where the goal is.	**Evaluation function** $f(n)$ — incorporates domain knowledge (estimates of cost to the goal). Expands the "most promising" node first.
Fringe data structure	**FIFO queue** — first-in, first-out. Nodes are processed in the order they are discovered.	**Priority queue** — nodes are dequeued in order of $f(n)$ (lowest first). The queue is dynamically reordered as new nodes are discovered.
Knowledge used	**None** — only the problem definition (initial state, goal test, successor function, step costs).	**Heuristic function** $h(n)$ — estimated cheapest cost from n to the goal. Domain-specific (e.g., straight-line distance, Manhattan distance).
Efficiency	**Expensive** for large state spaces — $O(b^d)$ time and space. Must explore all nodes at depth $d-1$ before checking depth d.	**Potentially much faster** — a good heuristic dramatically prunes the search space. But with a poor heuristic, can be as bad as uninformed search.
Completeness	**Complete** — guaranteed to find a solution if one exists (finite branching factor).	**Depends on the variant** — Greedy BFS is incomplete (can loop). A* is complete.
Optimality	**Optimal** only for uniform step costs.	**Depends on the variant** — Greedy BFS is not optimal. A* is optimal if h is admissible.
Examples	BFS, UCS (expand by $g(n)$), DFS, IDDFS	Greedy Best-First (expand by $h(n)$), A* (expand by $f(n) = g(n) + h(n)$), IDA*, SMA*

Key insight: "Uninformed" does *not* mean "stupid" — it means the algorithm works for **any** problem without modification. "Informed" means the algorithm exploits **domain-specific** knowledge to work more efficiently, but the heuristic must be designed for each problem. An uninformed algorithm needs no setup; an informed algorithm needs a clever heuristic.

Informed vs. Uninformed Search — The Fundamental Divide

Every search algorithm belongs to one of two families, distinguished by a single question: **Does it use domain-specific knowledge beyond the problem definition?**

Criterion	Uninformed (Blind) Search	Informed (Heuristic) Search
Knowledge used	Only the problem definition: initial state, goal test, successor function, step costs. **No estimate** of how close a state is to the goal.	Uses a **heuristic function** $h(n)$ — domain-specific knowledge estimating the cheapest cost from n to the goal.
How nodes are chosen	By **structural criteria** — depth (BFS, DFS, IDDFS), path cost so far (UCS), or symmetry (Bidirectional).	By an **evaluation function** $f(n)$ combining $g(n)$ and $h(n)$ — the algorithm steers toward promising-looking nodes.
Efficiency	**Exponential** in the worst case. Can explore vast portions of the state space with no guidance.	**Potentially far more efficient** — a good heuristic prunes large unpromising regions. With a perfect heuristic, A* expands only nodes on the optimal path.
Generality	**Universal** — works for *any* problem without modification. No domain knowledge needed. Zero setup cost.	**Problem-specific** — the heuristic must be designed (or learned) for each problem.
Optimality guarantee	UCS guarantees optimality for any non-negative costs; BFS/IDDFS only for uniform costs.	A* guarantees optimality if h is admissible ($h \leq h^*$). Greedy BFS gives no optimality guarantee.
When to use	Problem is new/unknown; no good heuristic exists; state space is small; or as a fallback.	Domain knowledge is available; an admissible heuristic can be defined; state space is large; optimality matters.
Examples	BFS, DFS, DLS, IDDFS, UCS, Bidirectional Search	Greedy BFS, A*, IDA*, RBFS, SMA*, Beam Search

The continuum — not a hard binary: Some algorithms blur the line.

- **UCS** uses $g(n)$ — this is "cost-aware" but not "goal-aware." It knows how much you've spent but has no idea how much remains. It is technically uninformed

because $g(n)$ is derived from the problem definition (step costs), not from domain knowledge.

- **Bidirectional Search** uses two simultaneous searches (forward from start, backward from goal). It is structurally informed — leveraging the fact that two smaller searches are cheaper than one large one — but uses no heuristic. It is formally uninformed.
- **A*** sits at the intersection: $f(n) = g(n) + h(n)$. The $g(n)$ part is uninformed (known costs); the $h(n)$ part is informed (domain estimate). This is why A* subsumes UCS (when $h(n)$=0) and Greedy (conceptually, when $g(n)$ is ignored).

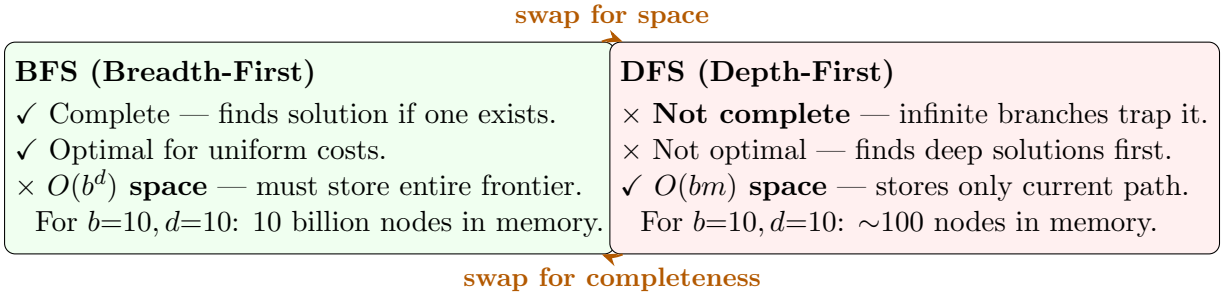
Progression of Uninformed Search Algorithms

Question: Discuss how uninformed search algorithms can be seen as an evolution of ideas, where each new algorithm is designed to address a limitation of an earlier one. Include the progression among: BFS, DFS, DLS, IDDFS, UCS, and Bidirectional Search.

Uninformed search algorithms did not emerge in isolation — they form a **logical progression**, each motivated by a specific weakness of its predecessor. Understanding this progression turns six seemingly disconnected algorithms into one coherent story.

Act I — The Foundational Trade-off: BFS vs. DFS

The two simplest algorithms represent opposite ends of a fundamental trade-off.



The dilemma: BFS guarantees the answer but needs impossible amounts of memory. DFS uses negligible memory but may never find the answer. The entire subsequent evolution is about **getting the best of both**.

Act II — DFS's First Fix: Depth-Limited Search (DLS)

Motivation	What DLS does	Remaining problem
DFS can descend **infinitely deep** on an infinite branch, never backtracking to find a shallow solution.	DLS adds a **depth cutoff** L: DFS stops at depth L and backtracks. This **prevents infinite descent** — the search is now bounded.	**How to choose L?** If $L < d$ (solution depth), DLS returns **failure** even though a solution exists. If $L \gg d$, DLS wastes time. DLS is **not complete** unless you already know d.

The contribution: DLS showed that adding a depth bound fixes DFS's completeness problem — but only if you know the right bound. The question becomes: *how to find d without knowing it in advance?*

Act III — Finding the Right Depth: Iterative Deepening DFS (IDDFS)

Motivation	What IDDFS does	Remaining problem
DLS fails if $L < d$ (misses solution) or wastes time if $L \gg d$. You don't know d in advance.	IDDFS runs DLS **repeatedly** with $L = 0, 1, 2, \ldots$ until the goal is found. This guarantees finding the shallowest solution — **completeness restored**.	**Non-uniform costs ignored.** IDDFS finds the shallowest goal (fewest steps), not the cheapest. If edge costs vary, the shortest path in steps may be expensive. Nodes near the root are re-expanded — overhead factor $\frac{b}{b-1}$.

The contribution: IDDFS solved the BFS-vs-DFS dilemma for uniform-cost problems: $O(bd)$ space (from DFS) + completeness + shallowest-solution optimality (from BFS). The repeated-work overhead is only $\sim$11% for b=10 — an acceptable price. **This is the best uninformed algorithm for uniform-cost problems with large state spaces.**

Act IV — Handling Non-Uniform Costs: Uniform-Cost Search (UCS)

Motivation	What UCS does	Remaining problem
BFS and IDDFS assume **all steps cost the same**. In the real world, actions have different costs (mountain roads cost more than highways). Neither shallowest-path algorithm can find the cheapest path with non-uniform costs.	UCS expands nodes by **lowest path cost** $g(n)$, not depth. It is **optimal for any non-negative edge costs** — the first time it expands the goal, the cheapest path is found.	**No guidance toward the goal.** UCS expands equally in *all* directions — it knows what you've spent but has **no idea how much remains**. In a large graph, UCS may explore enormous regions away from the goal.

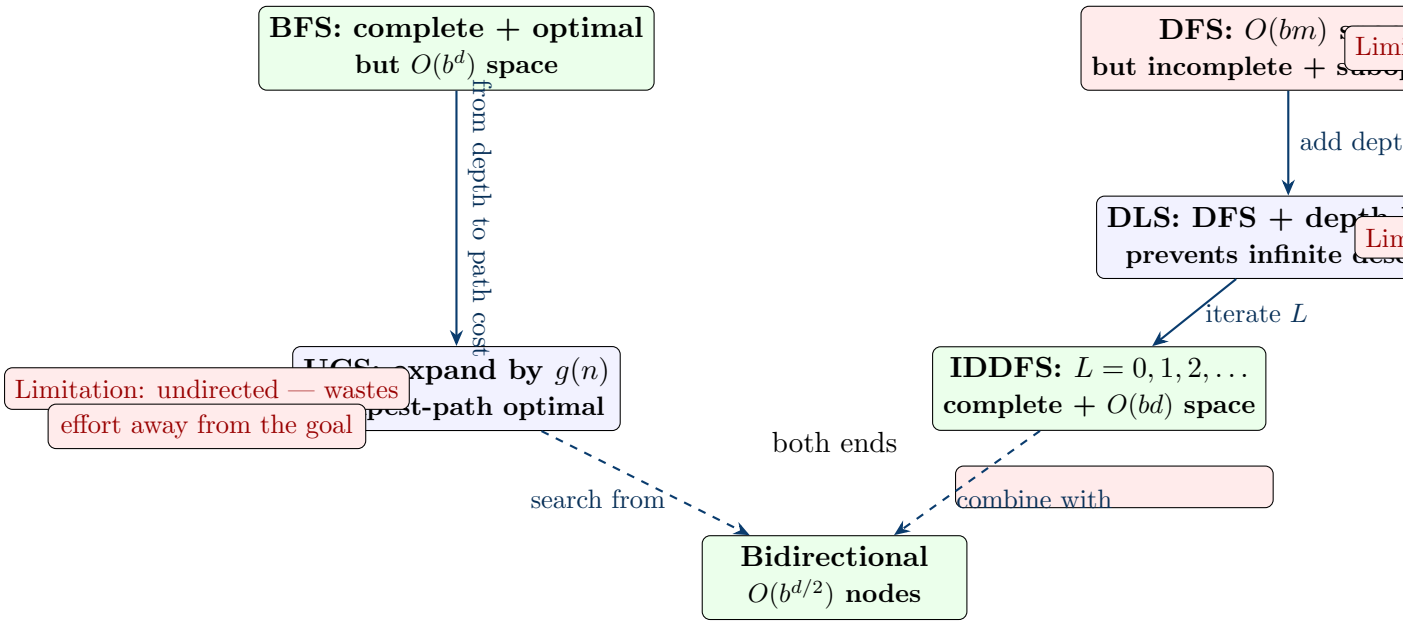
The contribution: UCS generalised optimality from "shallowest" to "cheapest." But it exposed a new problem: without knowledge of where the goal is, the algorithm is undirected — it wastes effort exploring in wrong directions. This limitation **directly motivates informed search** — add a heuristic $h(n)$ to estimate remaining distance, and UCS becomes A*.

Act V — Exploiting Symmetry: Bidirectional Search

Motivation	What Bidirectional does	Remaining problem
A single-direction search explores $O(b^d)$ nodes. But a search backward from the goal is equally valid — and two searches meeting in the middle each explore only $O(b^{d/2})$ nodes. This is an **exponential reduction**: $b^d \to 2 \cdot b^{d/2}$.	Run two simultaneous searches — one forward from start, one backward from goal(s). The solution is found when the two frontiers intersect. Can use BFS, UCS, or even A* as the underlying search method.	**Requires a reversible successor function.** The backward search needs a predecessor function — "what states lead *to* the goal?" This is not always easy to define. Also, checking frontier intersection at every step adds overhead. For some problems (theorem proving), the backward branching factor is much larger.

The contribution: Bidirectional search exploits problem **symmetry** — the fact that searching from the goal backward is just as valid as from the start forward. It is still uninformed, but achieves an exponential speedup through structural cleverness alone. Combined with IDDFS (bidirectional IDDFS) or A* (bidirectional A*), it becomes even more powerful.

The Complete Progression — Visual Summary



↑ Each arrow = a specific limitation being addressed

The Evolutionary Narrative (Exam Summary)

1. **BFS** — Gives us completeness and optimality, but at the impossible price of $O(b^d)$ memory. **Starting point.**

2. **DFS** — Flips the trade-off: $O(bm)$ memory is practical, but the algorithm may never find a solution. **Motivated by BFS's memory explosion.**

3. **DLS** — Fixes DFS's infinite-descent problem with a depth cutoff. But now you must guess the cutoff — guess too low and it reports failure when a solution exists. **Motivated by DFS's incompleteness.**

4. **IDDFS** — Eliminates the guesswork: $L = 0, 1, 2, \ldots$ until success. Restores completeness + shallowest-solution optimality while keeping $O(bd)$ space — the best of BFS and DFS. **Motivated by DLS's dependence on unknown L.**

5. **UCS** — Abandons depth for path cost $g(n)$. Finds the cheapest path regardless of non-uniform edge costs. But expands blindly in all directions. **Motivated by IDDFS/BFS's uniform-cost assumption.**

6. **Bidirectional Search** — Attacks exponential complexity directly: two $d/2$-depth searches replace one d-depth search. Orthogonal improvement — can be layered on top of BFS, UCS, or A*. **Motivated by single-direction search's $O(b^d)$ cost.**

The arc to informed search: The progression above solves structural problems (memory, completeness, cost-sensitivity) but never addresses the **core inefficiency** — none of these algorithms knows where the goal is. The next evolutionary step — **informed search** (Greedy, A*, IDA*) — adds a heuristic $h(n)$ that *estimates*

remaining distance. UCS with $h(n)$ becomes A*; IDDFS with $h(n)$ becomes IDA*. The progression continues, now with domain knowledge.

Heuristic Search — Introduction & the Best-First Family

What Makes a Search "Informed"?

An **informed** (or heuristic) search algorithm uses **problem-specific knowledge** beyond the problem definition to guide the search toward the goal more efficiently. This knowledge is encoded in a **heuristic function** $h(n)$.

Generic Best-First Search Algorithm:

1. Initialise the fringe (priority queue) with the start node.
2. **Loop:** Remove the node with the **lowest evaluation function** $f(n)$ from the fringe.
3. If that node is the goal, return the solution path.
4. Expand the node — generate all successors.
5. For each successor: compute f(successor), insert into the fringe (priority queue).
6. Repeat until the fringe is empty (failure).

The Best-First Search Family: The "family" is unified by the priority-queue fringe — the difference is **what** $f(n)$ measures:

Algorithm	Evaluation $f(n)$	Type	Key property
UCS	$f(n) = g(n)$	Uninformed	Optimal for any non-negative costs.
Greedy BFS	$f(n) = h(n)$	Informed	Fast but not optimal. Uses only heuristic.
A*	$f(n) = g(n) + h(n)$	Informed	Optimal if h is admissible. Best-known informed
IDA*	$f(n) = g(n) + h(n)$	Informed	Like A* but $O(bd)$ space. Uses iterative deepenir
RBFS	$f(n) = g(n) + h(n)$	Informed	Recursive Best-First — linear space, slightly mor
SMA*	$f(n) = g(n) + h(n)$	Informed	Simplified Memory-Bounded A* — uses all availa

Exam relevance: The CUET exam tests UCS, Greedy BFS, A*, and IDA*/IDDFS — these are the four you must be able to trace step-by-step on a given graph. RBFS and SMA* are mentioned in Russell & Norvig but have never appeared in a CUET past paper (2020–2024).

The Heuristic Function $h(n)$

Definition

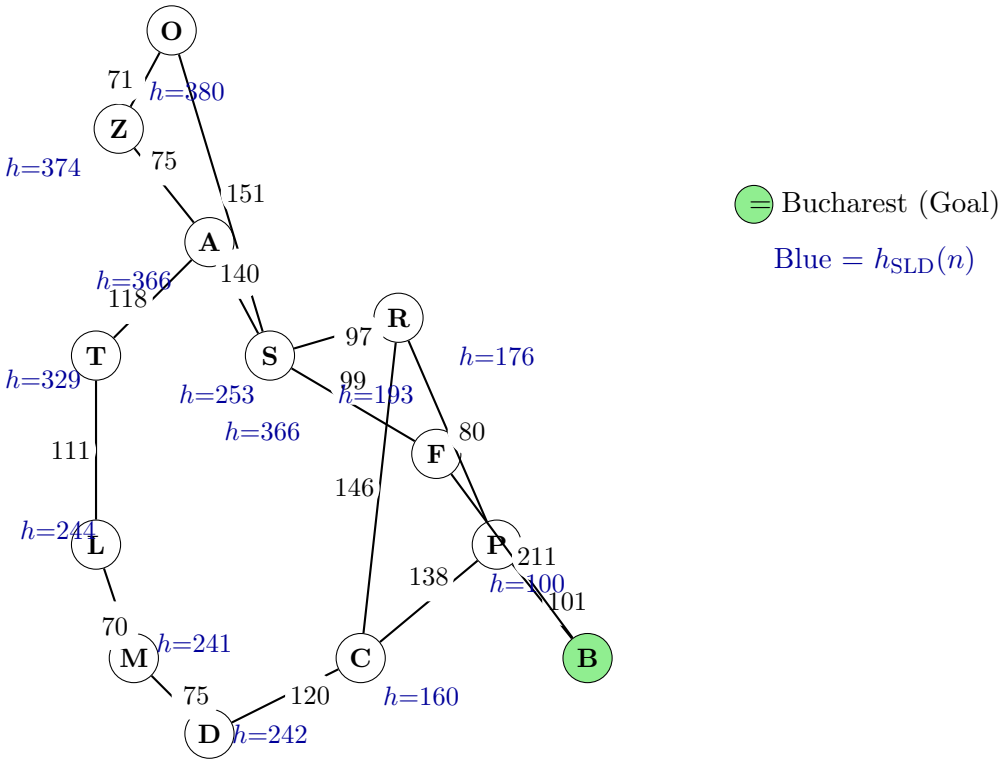
$$\boxed{h(n) = \text{estimated cheapest cost from node } n \text{ to a goal node}}$$

Key properties:

- $h(n)$ is **problem-specific** — you design it using domain knowledge. The same generic search code works for any problem, but $h(n)$ encodes what you know about *this* problem.
- $h(n) \geq 0$ for all n, and $h(G) = 0$ for any goal node G.
- $h(n)$ is **not** the action cost — it is an **estimate** of the remaining cost.
- A **perfect heuristic** satisfies $h(n) = h^*(n)$ for all n (where $h^*(n)$ is the true optimal cost). With a perfect heuristic, A* expands only nodes on optimal paths — it does zero wasted work.

Example — Romania Map with Straight-Line Distance Heuristic

The classic illustration from Russell & Norvig (Chapter 3). The problem: drive from **Arad** to **Bucharest** via the Romanian road network. The heuristic $h_{\text{SLD}}(n)$ is the **straight-line (as-the-crow-flies) distance** from city n to Bucharest — the shortest possible distance, ignoring roads.



City n	$h_{\text{SLD}}(n)$ (straight-line km)	True road distance $h^*(n)$
Arad	366	418
Bucharest	0	0
Craiova	160	238
Dobreta	242	360
Fagaras	176	211
Lugoj	244	354
Mehadia	241	353
Oradea	380	531
Pitesti	100	101
Rimnicu	193	193
Sibiu	253	278
Timisoara	329	469
Zerind	374	449

How h_{SLD} works:

- For **any** two points on a map, the straight-line distance is a **lower bound** on the road distance — you cannot drive shorter than a straight line.
- Therefore $h_{\text{SLD}}(n) \leq h^*(n)$ for every city — the heuristic is **admissible**.
- **Check:** Arad: $366 \leq 418$ ✓. Pitesti: $100 \leq 101$ (nearly perfect!). Rimnicu: $193 = 193$ (perfect!). Fagaras: $176 \leq 211$ ✓.
- **No city has** $h > h^*$ — the heuristic never overestimates, guaranteeing A* optimality on this problem.
- The heuristic is **not perfect** — Fagaras looks deceptively close (176) but the actual road is 211 (the Carpathian mountains force a winding route). This is exactly why Greedy fails on this problem.

Greedy Best-First Search — Algorithm, Example, Properties

Algorithm

Greedy Best-First Search uses **only the heuristic** as its evaluation function:

$$\boxed{f(n) = h(n)}$$

At every step, expand the fringe node with the **lowest heuristic value** $h(n)$. The fringe is a **min-priority queue sorted by** $h(n)$. Path cost $g(n)$ is **completely ignored**.

Romania Trace — How Greedy Finds a Suboptimal Path

Problem: drive from **Arad** to **Bucharest** using h_{SLD}.

Step	Expanded	$h(n)$	Fringe (sorted by h)
1	**Arad**	366	S[253], T[329], Z[374] Neighbours: Sibiu(253), Timisoara(329), Zerind(374). Sibiu looks closest.
2	**Sibiu**	253	F[176], R[193], T[329], A[366], Z[374], O[380] Fagaras has h=176 — lowest! Greedy rushes toward it.
3	**Fagaras**	176	B[0], R[193], T[329], A[366], Z[374], O[380] Bucharest has h=0 — it is the goal!
4	**Bucharest**	0	**GOAL FOUND**

Greedy path: Arad $\xrightarrow{140}$ Sibiu $\xrightarrow{99}$ Fagaras $\xrightarrow{211}$ Bucharest $\Rightarrow$ **Cost = 450**
Optimal path (A* finds this; Greedy misses it): Arad $\xrightarrow{140}$ Sibiu $\xrightarrow{80}$ Rimnicu $\xrightarrow{97}$ Pitesti $\xrightarrow{101}$ Bucharest $\Rightarrow$ **Cost = 418**

Why Greedy failed — the root cause:
At Sibiu, Greedy compared two routes eastward:

- **Fagaras:** h=176 — looks just 176 km from Bucharest as the crow flies!
- **Rimnicu:** h=193 — looks slightly further.

Greedy chose Fagaras because $176 < 193$, **completely ignoring** that:

- The road from Fagaras to Bucharest is 211 km (mountains force a winding route — h underestimates by 35 km).
- The Rimnicu → Pitesti → Bucharest route costs $80 + 101 = 181$ km total — h for Rimnicu is only off by 0 (it's perfect!) and Pitesti is off by just 1.

Moral: A low $h(n)$ can be **deceptive**. A node that looks close to the goal may have a very expensive road ahead. Greedy's refusal to track how much it has already spent ($g(n)$) is its fatal flaw.

Properties of Greedy Best-First Search

Property	Status	Explanation
Completeness	× **Not complete**	Can get stuck in infinite loops — if t heuristic guides it into a cycle, it m never escape. A closed set fixes this b costs $O(b^d)$ memory. On finite grap with a closed set, it is complete.
Optimality	× **Not optimal**	Ignores $g(n)$ — will take a cheap-looki path even if the total cost is high. Rom nia: cost 450 vs. optimal 418.
Time	$O(b^m)$ worst case	Can explore the entire tree with a po heuristic. With a good heuristic, e tremely fast — Romania: only 4 expa sions.
Space	$O(b^m)$	Stores all generated nodes in the priori queue. Good heuristic ⇒ fewer nodes ge erated.
Sensitivity	**Very high** — entirely dependent on h quality.	With perfect $h = h^*$: optimal in $O(d)$ e pansions. With $h = 0$ everywhere: dege erates to random selection from the fring

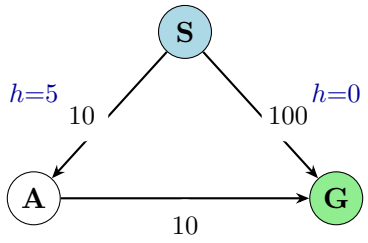
Greedy vs. A* — Direct Comparison

Both are **informed search algorithms** using a priority queue. The critical difference is **what they optimise**:

$$\text{Greedy: } f(n) = h(n) \qquad \text{A*: } f(n) = g(n) + h(n)$$

Criterion	Greedy Best-First	A* Search
Priority	Lowest $h(n)$ only — "Which node *looks* closest to the goal?"	Lowest $g(n) + h(n)$ — "Which node gives the cheapest estimated total path?"
Completeness	× Not complete (can loop without closed set)	✓ **Complete** on finite graphs
Optimality	× **Not optimal** — ignores past cost	✓ **Optimal** if h is admissible ($h \leq h^*$)
Time	$O(b^m)$ worst case; very fast with good h	$O(b^d)$ worst case; $O(d)$ with perfect h
Space	$O(b^m)$ — stores fringe	$O(b^d)$ — stores fringe (A*'s main practical weakness)
Heuristic req.	None (works with any h)	**Must be admissible** for optimality; consistent for efficiency on graphs
Romania result	4 expansions, cost 450 (suboptimal)	6 expansions, cost 418 (optimal)
Use when	Speed > optimality; excellent heuristic; large state space	Optimality required; admissible heuristic; state space fits in memory

The core intuition — a simple counterexample where Greedy loses:



G is the goal. S connects directly to G (cost 100). Or S→A→G (cost $10 + 10 = 20$).

Greedy: At S: h(A)=5, h(G)=0. Greedy picks G (lowest h), finds path S→G at cost **100**. Done. **A*:** At S: f(A)$= 10 + 5 = 15$, f(G)$= 100 + 0 = 100$. A* expands A first, then A→G at cost **20**. **Optimal.**
Greedy was fooled — G looks perfect (h=0, it *is* the goal!), but reaching it directly costs 100. Path A looks slightly less perfect (h=5) but costs only 10 to reach and only 10 more to G. A* accounts for both g and h; Greedy is blind to g.

Examination 2024 — Q2 (9 Marks)

Problem Data

States: A (start), B, C, D, E, F, H, J, K, G (goal). All roads bidirectional.

Table 1: Road Connections and Step Costs

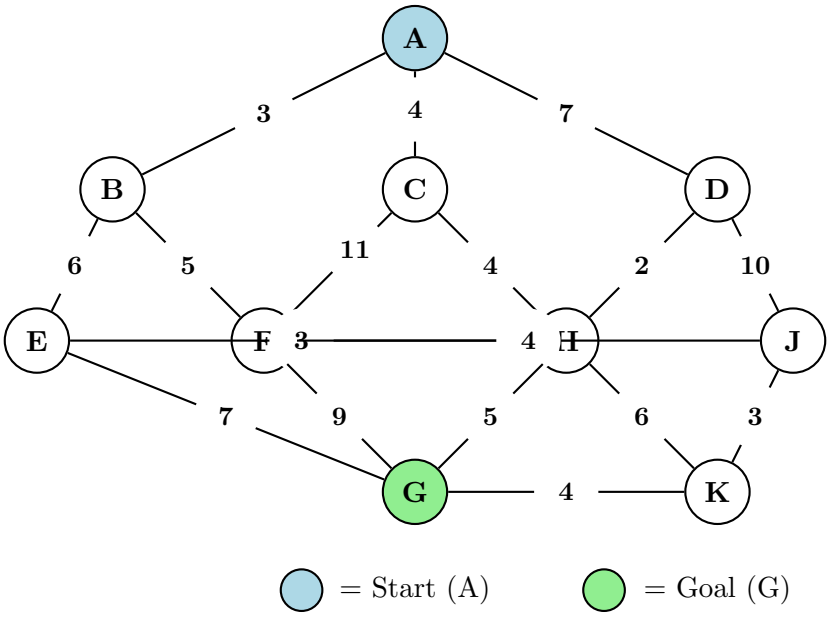
From	To	Cost
A	B	3
A	C	4
A	D	7
B	E	6
B	F	5
C	F	11
C	H	4
D	H	2
D	J	10
E	G	7
E	H	3
F	G	9
F	J	4
H	G	5
H	K	6
J	K	3
K	G	4

Table 2: Heuristic Estimates $h(n)$ to Goal G

Node	h(n)
A	11
B	10
C	7
D	6
E	6
F	6
H	4
J	5
K	3
G	0

Part a(i) — State-Space Graph [1 mark]

Question: Using Table 1, draw the state-space graph clearly labelling all nodes and edge costs.



Note: The edge E–H is curved upward and F–J is curved downward to avoid visual ambiguity where nodes share the same y-coordinate. All edges are **bidirectional** (undirected).

Part a(ii) — Search Algorithm Traces [5 marks]

Question: Show search trees for (1) UCS, (2) Greedy Best-First, (3) A*, (4) IDDFS from A to G. At every step: show node being expanded and fringe contents in order of selection. For IDDFS: show depth limit per iteration. Ties broken alphabetically.

Algorithm 1 — Uniform-Cost Search (UCS)

Key idea: Always expand the node with the **lowest path cost** $g(n)$ from the start. The fringe is a **min-priority queue sorted by** $g(n)$. A node is closed (never re-expanded) once popped. If a *cheaper* path to a fringe node is found, its cost is updated (replaced).

Fringe notation: Node[g] sorted ascending by g.

Step	Node Expanded	g(n)	Fringe after expansion (sorted by g)
1	**A**	0	B[3], C[4], D[7] Neighbours of A: B(g=3), C(g=4), D(g=7).
2	**B**	3	C[4], D[7], F[8], E[9] A already closed. $B\to F$: g=8; $B\to E$: g=9. Both new.
3	**C**	4	D[7], F[8], H[8], E[9] $C\to F$: g=15 > 8 already in fringe ⇒ keep F[8]. $C\to H$: g=8 new.
4	**D**	7	F[8], H[8], E[9], J[17] $D\to H$: g=9 > 8 in fringe ⇒ keep H[8]. $D\to J$: g=17 new. Tie F,H: **F before H** (alphabetical).
5	**F**	8	H[8], E[9], J[12], G[17] $F\to G$: g=17 new. $F\to J$: g=12 < 17 in fringe ⇒ **update** J[12].
6	**H**	8	E[9], J[12], G[13], K[14] $H\to E$: g=11 > 9 in fringe ⇒ keep E[9]. $H\to G$: g=13 < 17 ⇒ **update** G[13]. $H\to K$: g=14 new.
7	**E**	9	J[12], G[13], K[14] $E\to G$: g=16 > 13 in fringe ⇒ keep G[13].
8	**J**	12	G[13], K[14] $J\to K$: g=15 > 14 in fringe ⇒ keep K[14].
9	**G**	13	**GOAL FOUND**

UCS Solution: $A \xrightarrow{4} C \xrightarrow{4} H \xrightarrow{5} G$ **Total cost:** $4+4+5=$ **13**
Nodes expanded (in order): A, B, C, D, F, H, E, J, G (9 nodes total)
UCS **guarantees the optimal path** because it always expands the cheapest unexplored node, never re-expands a closed node, and path costs are non-negative.

Algorithm 2 — Greedy Best-First Search

Key idea: Always expand the node with the **lowest heuristic estimate** $h(n)$ to the goal. The fringe is sorted by $h(n)$ (ascending). Path cost $g(n)$ is *completely ignored*. Greedy is fast but not guaranteed to find the optimal path.

Fringe notation: Node[h] sorted ascending by h.

Step	Node Expanded	h(n)	Fringe after expansion (sorted by h)
1	**A**	11	D[6], C[7], B[10] Neighbours B(h=10), C(h=7), D(h=6). Sorted by h: D, C, B.
2	**D**	6	H[4], J[5], C[7], B[10] D's neighbours excl. A: H(h=4), J(h=5).
3	**H**	4	G[0], K[3], J[5], E[6], C[7], B[10] H's neighbours excl. D: C(h=7, already in fringe—skip), E(h=6), G(h=0), K(h=3).
4	**G**	0	**GOAL FOUND**

Greedy Solution: $A \xrightarrow{7} D \xrightarrow{2} H \xrightarrow{5} G$ **Total cost:** $7+2+5=$ **14** (not optimal!)

Nodes expanded: A, D, H, G (only 4 nodes — very fast)

Greedy expanded just 4 nodes versus UCS's 9, but found a **suboptimal path** (cost 14 vs optimal 13). It followed the heuristic blindly: D had the lowest h from A, leading it down the D–H–G route instead of the cheaper A–C–H–G route.

Algorithm 3 — A* Search

Key idea: Expand the node with the **lowest evaluation function** $f(n) = g(n) + h(n)$, where $g(n)$ = exact cost from start and $h(n)$ = heuristic estimate to goal. The fringe is sorted by $f(n)$ (ascending). Ties in f broken alphabetically. If a cheaper path to a fringe node is found, update its entry.

Fringe notation: Node$[g, h, f]$ sorted ascending by f.

Step	Node	g	h	f	Fringe after expansion (sorted by f)
1	**A**	0	11	11	C[4,7,**11**], B[3,10,**13**], D[7,6,**13**] Tie B,D at f=13: B first.
2	**C**	4	7	11	H[8,4,**12**], B[3,10,**13**], D[7,6,**13**], F[15,6,**21**] $C \to H$: g=8, f=12 new. $C \to F$: g=15, f=21 new.
3	**H**	8	4	12	B[3,10,**13**], D[7,6,**13**], G[13,0,**13**], E[11,6,**17**], K[14,3,**17**], F[15,6,**21**] $H \to D$: f=16 > 13, keep. $H \to G$: f=13 new. $H \to E$: f=17 new. $H \to K$: f=17 new. Tie B,D,G at f=13: **B first**.
4	**B**	3	10	13	D[7,6,**13**], G[13,0,**13**], F[8,6,**14**], E[9,6,**15**], K[14,3,**17**] $B \to F$: g=8, f=14 < 21 ⇒ **update** F. $B \to E$: g=9, f=15 < 17 ⇒ **update** E. Tie D,G at f=13: **D first**.
5	**D**	7	6	13	G[13,0,**13**], F[8,6,**14**], E[9,6,**15**], K[14,3,**17**], J[17,5,**22**] $D \to J$: g=17, f=22 new.
6	**G**	13	0	13	**GOAL FOUND**

A* Solution: $A \xrightarrow{4} C \xrightarrow{4} H \xrightarrow{5} G$ **Total cost:** $4+4+5 = \mathbf{13}$ (optimal!)
Nodes expanded: A, C, H, B, D, G (6 nodes — fewer than UCS's 9)
A* is both **complete** and **optimal** when the heuristic is *admissible* ($h(n) \leq h^*(n)$, i.e. it never overestimates). It is more efficient than UCS because $h(n)$ guides the search toward promising nodes, pruning unpromising directions early.

Algorithm 4 — Iterative Deepening DFS (IDDFS)

Key idea: Perform DFS with a depth cutoff $L = 0, 1, 2, \ldots$, restarting from the root each time, until the goal is found. The "fringe" is a **stack** (LIFO). Children are pushed in *reverse* alphabetical order so they are *popped* (and expanded) in alphabetical order. Nodes on the **current path** (ancestors) are not revisited to avoid infinite loops.

Stack notation: Leftmost entry = top of stack (next to be popped). Depth shown in parentheses.

Iteration $L = 0$:

Step	Action	Stack (top ← left)
1	Pop **A**(d=0). $A \neq G$. At depth limit — cannot expand.	[ ]
	No solution at $L = 0$.	

Iteration $L = 1$**:**

Step	Action	Stack
1	Pop **A**(d=0). Expand: push D,C,B (B on top).	B(1), C(1), D(1)
2	Pop **B**(d=1). $B \neq G$. At depth limit.	C(1), D(1)
3	Pop **C**(d=1). $C \neq G$. At depth limit.	D(1)
4	Pop **D**(d=1). $D \neq G$. At depth limit.	[ ]
	No solution at $L = 1$.	

Iteration $L = 2$**:**

Step	Action	Stack
1	Pop **A**(d=0). Expand: B,C,D.	B(1), C(1), D(1)
2	Pop **B**(d=1). Expand (excl. A): E,F. Push F,E.	E(2), F(2), C(1), D(1)
3	Pop **E**(d=2). $E \neq G$. At limit.	F(2), C(1), D(1)
4	Pop **F**(d=2). At limit.	C(1), D(1)
5	Pop **C**(d=1). Expand (excl. A): F,H. Push H,F.	F(2), H(2), D(1)
6	Pop **F**(d=2). At limit.	H(2), D(1)
7	Pop **H**(d=2). At limit.	D(1)
8	Pop **D**(d=1). Expand (excl. A): H,J. Push J,H.	H(2), J(2)
9	Pop **H**(d=2). At limit.	J(2)
10	Pop **J**(d=2). At limit.	[ ]
	No solution at $L = 2$.	

Iteration $L = 3$**:**

Step	Action	Stack
1	Pop **A**(d=0). Expand: B,C,D.	B(1), C(1), D(1)
2	Pop **B**(d=1). Expand (excl. A): E,F.	E(2), F(2), C(1), D(1
3	Pop **E**(d=2). Expand (excl. B): G,H. Push H first then G (G on top).	G(3), H(3), F(2), C(1
4	Pop **G**(d=3). G = GOAL.	**GOAL FOUND**

IDDFS Solution: $A \xrightarrow{3} B \xrightarrow{6} E \xrightarrow{7} G$ **Depth: 3** **Cost:** $3+6+7=\mathbf{16}$

IDDFS found the solution at depth-limit $L=3$ in just 4 node expansions during the final iteration.

Important distinction: IDDFS finds the *shallowest* path (fewest hops), **not** the lowest-cost path. The optimal path A→C→H→G also has depth 3, but alphabetical DFS expansion reaches G via B→E→G first. For non-uniform step costs, IDDFS is **not optimal**. It is however **complete** — it will always find a solution if one exists.

Why IDDFS over BFS? IDDFS uses only $O(bL)$ *space* (a single stack), whereas BFS requires $O(b^L)$ space to store all frontier nodes. The repeated work is a constant factor overhead.

Algorithm Comparison Summary:

Algorithm	Path Found	Cost	Nodes Expanded	Optimal?
UCS	A → C → H → G	13	9 (all iterations)	**Yes**
Greedy	A → D → H → G	14	4	No
A*	A → C → H → G	13	6	**Yes**
IDDFS	A → B → E → G	16	4 (final iter.)	No (non-uniform costs)

Part b — Problem Formulations [3 marks]

Question: Give the initial state, goal test, operators, and path cost function for each problem.

(i) Robot Vacuum Cleaning Problem (Indoor Environment)

State: A pair $(l,\ D)$ where l is the robot's current room and $D \subseteq \{\text{all rooms}\}$ is the set of dirty rooms. *Example:* (Kitchen, {Bedroom, Lounge}).

Initial state: (start room, {all currently dirty rooms}).

Goal test: $D = \emptyset$ (no dirty rooms remain).

Operators:

1. Move($r_i \to r_j$): Move from room r_i to adjacent room r_j (changes location; D unchanged).
2. Suck(): Vacuum the current room (removes current room from D if dirty).

Static obstacles (furniture) constrain which rooms are adjacent.

Path cost: Total energy used: each Move costs c_{move} energy units; each Suck costs c_{suck} units. Goal: minimise total energy.

Informed vs. Uninformed: Informed search (A*) is more suitable. A strong heuristic $h(n) = |D| \times \overline{d}$ (number of remaining dirty rooms multiplied

by average travel cost between rooms) estimates remaining work without overestimating. With many rooms, uninformed BFS/UCS must explore exponentially many (l, D) state combinations blindly. A* with a good heuristic dramatically prunes this search space.

(ii) Water Jug Problem (5L and 3L Jugs)

State: A pair $(\ x,\ y\ )$ where $x \in \{0, 1, 2, 3, 4, 5\}$ is water (litres) in the 5L jug and $y \in \{0, 1, 2, 3\}$ is water in the 3L jug.

Initial state: $(0,\ 0)$ — both jugs empty.

Goal test: $x = 4$ (exactly 4 litres in the 5L jug). The 3L jug cannot hold 4L, so only $x = 4$ satisfies the goal.

Operators:

1. **Fill 5L:** $(x, y) \to (5,\ y)$
2. **Fill 3L:** $(x, y) \to (x,\ 3)$
3. **Empty 5L:** $(x, y) \to (0,\ y)$
4. **Empty 3L:** $(x, y) \to (x,\ 0)$
5. **Pour 5L → 3L:** $(x, y) \to (x{-}d,\ y{+}d)$ where $d = \min(x,\ 3{-}y)$
6. **Pour 3L → 5L:** $(x, y) \to (x{+}d,\ y{-}d)$ where $d = \min(y,\ 5{-}x)$

Path cost: Number of actions performed (1 per action), minimised to find the shortest sequence.

State space: **FINITE.** The 5L jug holds integer amounts $\{0, 1, 2, 3, 4, 5\}$ (6 values) and the 3L jug holds $\{0, 1, 2, 3\}$ (4 values), giving at most $6 \times 4 = 24$ states. Not all are reachable, but the space is strictly bounded.

One valid solution path (6 steps):

$$(0,0) \xrightarrow{\text{Fill 5L}} (5,0) \xrightarrow{\text{Pour 5}\to\text{3}} (2,3) \xrightarrow{\text{Empty 3L}} (2,0) \xrightarrow{\text{Pour 5}\to\text{3}} (0,2) \xrightarrow{\text{Fill 5L}} (5,2) \xrightarrow{\text{Pour 5}\to\text{3}} (4$$

Examination 2023 — Q2 (9 Marks)

Problem Data

Table 1: State Transitions and Step Costs

State	Next	Cost
A	B	4
A	C	1
B	D	3
B	E	8
C	C	0
C	D	2
C	F	6
D	C	2
D	E	4
E	G	2
F	G	8

Table 2: Heuristic Estimates $h(n)$

Node	h(n)
A	8
B	8
C	6
D	5
E	1
F	4
G	0

Important: This is a **directed** graph — edges go one way only (State → Next). Note the self-loop C→C with cost 0 — this is a degenerate edge and should be ignored during search (it does not change the state or advance toward the goal). Also note the edge D→C(2): returning from D to C is allowed since the graph is directed — C is reachable from both A and D. No reverse edges from B→A, C→A, D→B, etc. exist unless listed in the table above.

Part a — Problem Formulations [2 marks]

Question: Give the initial state, goal test, operators, and path cost for each problem.

(i) Travelling Salesman Problem (TSP)

State: A pair $(\ell,\ V)$ where ℓ is the current city and $V \subseteq \{\text{all } N \text{ cities}\}$ is the set of *unvisited* cities. *Example midway:* (Dhaka, {Chittagong, Sylhet}) means we are in Dhaka and still need to visit Chittagong and Sylhet.

Initial state: (start city, {all $N-1$ other cities}).

Goal test: $\ell =$ start city **and** $V = \emptyset$ (returned to start after visiting all cities exactly once).

Operators: Travel($\text{city}_i \to \text{city}_j$) where $\text{city}_j \notin V$ or ($V{=}\emptyset$ and $\text{city}_j{=}$start). Each move removes city_j from V.

Path cost: Sum of distances (or costs) of all travelled roads. Goal: find the tour with **minimum total cost**.

Complexity note: With N cities there are $(N-1)!/2$ distinct tours — factorial growth. TSP is NP-hard, so exhaustive search is infeasible for $N > 20$. Heuristic search (A*) with an admissible heuristic (e.g. minimum spanning tree of remaining cities) can find optimal or near-optimal tours for moderate N.

(ii) Missionaries & Cannibals

State: A triple $(M_L,\ C_L,\ B)$ where M_L = missionaries on left bank (0–3), C_L = cannibals on left bank (0–3), $B \in \{L, R\}$ = boat position. The right bank has $3-M_L$ missionaries and $3-C_L$ cannibals.

Initial state: $(3,\ 3,\ L)$ — all six people and the boat on the left bank.

Goal test: $(0,\ 0,\ R)$ — all six people and the boat on the right bank.

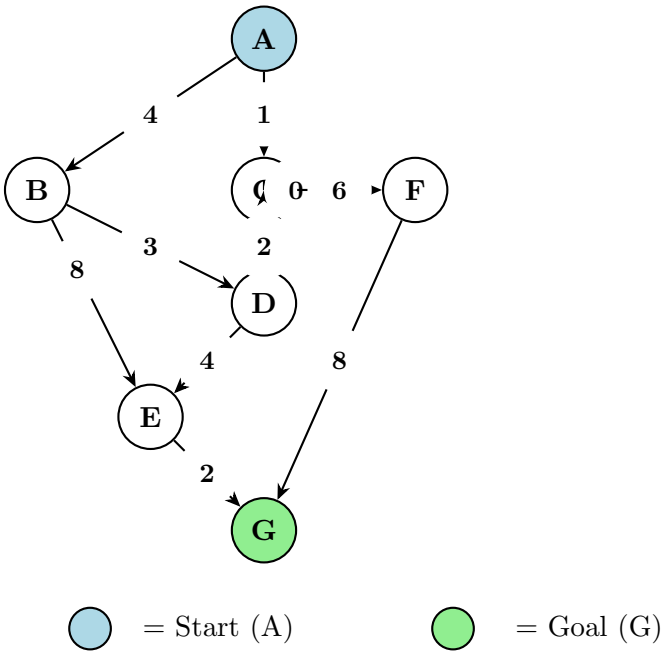
Operators: Move k missionaries and j cannibals across the river **by boat**, where $1 \le k+j \le 2$ (boat capacity 2), and the resulting state must be **legal**: on both banks, $M \ge C$ *or* $M = 0$ (missionaries never outnumbered by cannibals on either side).

Possible boat loads: (1M,0C), (2M,0C), (0M,1C), (0M,2C), (1M,1C).

Path cost: Number of boat trips (1 per crossing). Goal: find a sequence with **minimum trips**.

State space size: Without constraints: $4\times4\times2 = 32$ states. After legality filtering, only about 16–20 are valid. The problem has a known shortest solution of **11 crossings**.

Part b(i) — State-Space Diagram [0.5 mark]



Notes on the graph structure:

- C→C(0) is a zero-cost self-loop — degenerate; ignored in search.
- D→C(2) creates a directed 2-cycle C ⇌ D (C→D costs 2, D→C costs 2).
- B has two outgoing edges: B→D(3) and B→E(8). B→E is an expensive direct path.
- The cheap route to the goal passes through D→E(4)→G(2).
- From A, the best first step is C (cost 1), not B (cost 4).

Part b(ii) — Search Algorithm Traces [4.5 marks]

Question: Trace UCS, Greedy search, and A* from A (initial) to G (goal). At each step: show the node being expanded and the fringe contents. Ties broken alphabetically.

Algorithm 1 — Uniform-Cost Search (UCS)

Key idea: Expand the node with the **lowest path cost** $g(n)$. Fringe is a min-priority queue sorted by $g(n)$. Once a node is expanded (closed), it is never re-expanded. If a cheaper path to a fringe node is found, update its cost.

Fringe notation: Node[g] sorted ascending by g. **Closed set:** {nodes already expanded}.

Step	Node	g	Fringe after expansion (sorted by g)
1	**A**	0	C[**1**], B[**4**] A→C: g=1. A→B: g=4. Both new.
2	**C**	1	D[**3**], B[**4**], F[**7**] C→C: self-loop, ignore. C→D: g=3 new. C→F: g=7 new.
3	**D**	3	B[**4**], E[**7**], F[**7**] D→C: g=5, C already closed ⇒ skip. D→E: g=7 new.
4	**B**	4	E[**7**], F[**7**] B→D: g=7, D already closed ⇒ skip. B→E: g=12 > E[7] in fringe ⇒ keep E[7].
5	**E**	7	F[**7**], G[**9**] Tie E[7], F[7]: **E before F** (alphabetical). E→G: g=9 new.
6	**F**	7	G[**9**] F→G: g=15 > G[9] in fringe ⇒ keep G[9].
7	**G**	9	**GOAL FOUND**

UCS Solution: $A \xrightarrow{1} C \xrightarrow{2} D \xrightarrow{4} E \xrightarrow{2} G$ **Total cost:** $1+2+4+2 = \mathbf{9}$
Nodes expanded (in order): A, C, D, B, E, F, G (7 nodes total)
UCS guarantees the optimal path because it explores in order of increasing $g(n)$ and step costs are non-negative.

Algorithm 2 — Greedy Best-First Search

Key idea: Expand the node with the **lowest heuristic** $h(n)$. Fringe sorted by h ascending. Path cost $g(n)$ is *completely ignored*. This is fast but not guaranteed optimal.
Fringe notation: Node[h] sorted ascending by h.

Step	Node	h	Fringe after expansion (sorted by h)
1	**A**	8	C[**6**], B[**8**] $A\to C$: h=6. $A\to B$: h=8.
2	**C**	6	F[**4**], D[**5**], B[**8**] $C\to F$: h=4. $C\to D$: h=5. $C\to C$ ignored.
3	**F**	4	G[**0**], D[**5**], B[**8**] $F\to G$: h=0. Goal in fringe with lowest h.
4	**G**	0	**GOAL FOUND**

Greedy Solution: $A \xrightarrow{1} C \xrightarrow{6} F \xrightarrow{8} G$ **Total cost:** $1+6+8 = \mathbf{15}$ (suboptimal!)
Nodes expanded: A, C, F, G (only 4 nodes — extremely fast)
Greedy blindly chased the heuristic: from C, F has h=4 (lowest), so it went C→F, but F→G costs 8 — the most expensive path to the goal. It completely missed the cheap C→D→E→G route (cost 9) because D's h=5 looked slightly worse than F's h=4.

Algorithm 3 — A* Search

Key idea: Expand the node with the **lowest** $f(n) = g(n) + h(n)$. Fringe sorted by f ascending. Ties in f broken alphabetically. Closed set prevents re-expansion.
Fringe notation: Node[g, h, f] sorted ascending by f.

Step	Node	g	h	f	Fringe after expansion (sorted by f)
1	**A**	0	8	8	C[1,6,**7**], B[4,8,**12**] $A \to C$: f=1+6=7. $A \to B$: f=4+8=12.
2	**C**	1	6	7	D[3,5,**8**], F[7,4,**11**], B[4,8,**12**] $C \to D$: g=3, f=8. $C \to F$: g=7, f=11. C→C ignored.
3	**D**	3	5	8	E[7,1,**8**], F[7,4,**11**], B[4,8,**12**] $D \to C$: C closed ⇒ skip. $D \to E$: g=7, f=8. Tie D(f=8) and E(f=8): D already expanded, E is in fringe.
4	**E**	7	1	8	G[9,0,**9**], F[7,4,**11**], B[4,8,**12**] $E \to G$: g=9, f=9.
5	**G**	9	0	9	**GOAL FOUND**

A* Solution: $A \xrightarrow{1} C \xrightarrow{2} D \xrightarrow{4} E \xrightarrow{2} G$ **Total cost:** $1+2+4+2 = \mathbf{9}$ (optimal!)

Nodes expanded: A, C, D, E, G (only 5 nodes — fewer than UCS's 7)

A* found the optimal path and expanded **only 5 nodes** compared to UCS's 7, because the heuristic $h(n)$ guided it away from the expensive B and F branches. Note that **B was never expanded** — its f=12 was always worse than the alternatives ahead of it in the fringe.

Algorithm Comparison for 2023 Graph:

Algorithm	Path Found	Cost	Nodes Expanded	Optimal?
UCS	A → C → D → E → G	**9**	7	**Yes**
Greedy	A → C → F → G	15	4	No
A*	A → C → D → E → G	**9**	5	**Yes**

Why does A* outperform UCS here? The heuristic $h(n)$ provides a useful lower bound. After expanding A and C, A* sees that D has f=8 (promising — $h(D)$=5 is close to the true remaining cost of 6) while B has f=12 (unpromising — $h(B)$=8 overestimates its potential). A* expands D next and finds the optimal path without ever touching B. UCS, lacking a heuristic, expands B unnecessarily at step 4 before reaching the goal.

Admissibility check: Is $h(n)$ admissible? Let's verify:

- $h(A)$=8 ≤ $h^*(A)$=9 ✓
- $h(C)$=6 ≤ $h^*(C)$=8 ✓
- $h(D)$=5 ≤ $h^*(D)$=6 ✓
- $h(E)$=1 ≤ $h^*(E)$=2 ✓
- $h(B)$=8 ≤ $h^*(B)$=5? **No!** $h^*(B) = \min(3+4+2,\ 8+2) = 9$. Wait — $B \to D \to E \to G$ = 3+4+2 = 9, so $h(B)$=8 < 9 ✓. And $B \to E \to G$ = 8+2 = 10. So $h^*(B)$=9, $h(B)$=8 is admissible.

- $h(F)=4 \leq h^*(F)=8$ ✓

Yes, the heuristic is admissible — $h(n) \leq h^*(n)$ for all nodes. This is why A* found the optimal solution.

Part c — IDDFS: Synthesising Benefits of BFS and DFS [2 marks]

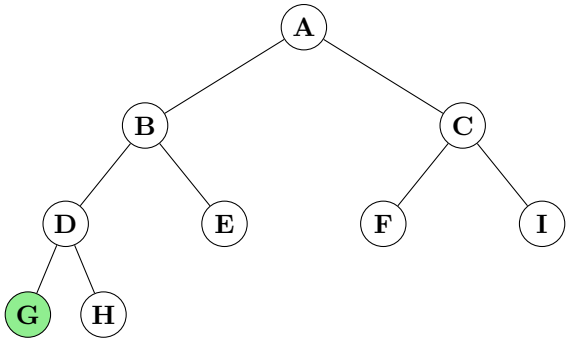
Question: Illustrate with an appropriate example how IDDFS synthesises the benefits of both BFS and DFS — achieving optimality, completeness, and linear space complexity. Nevertheless, time complexity deteriorates in comparison to both BFS and DFS.

The Core Idea

IDDFS (Iterative Deepening Depth-First Search) runs DFS repeatedly with an increasing depth cutoff: first L=0, then L=1, L=2, ..., until the goal is found. At each iteration, the search is a standard depth-limited DFS from the root.

Illustrative Example

Consider the following binary tree where G (shaded) is the shallowest goal at depth 3:



IDDFS Execution:

L	Nodes visited (DFS order)	Outcome
0	A	A ≠ G. No solution at depth 0.
1	A, B, C	Neither B nor C is G. No solution at depth 1.
2	A, B, D, E, C, F, I	No goal at depth 2.
3	A, B, D, **G** ← **GOAL FOUND**	**Found!** Shallowest goal at depth 3.

How IDDFS Inherits Each Algorithm's Strengths

Property	BFS	DFS	IDDFS
Completeness	✓ Complete (finds solution if one exists)	× Incomplete (may loop forever on infinite branches)	✓ **Complete** — like Depth-limited DFS a infinite loops becaus backtracks at the cutof
Optimality	✓ Optimal for uniform step costs (finds shallowest solution)	× Not optimal (may find a deep solution first)	✓ **Optimal for unif costs** — like BFS. cause it explores all n at depth $d-1$ before ar depth d, the first solu found is the shallowest.
Space	× $O(b^d)$ — stores entire frontier. For b=10, d=10, that's 10 billion nodes.	✓ $O(bm)$ — stores only one path + siblings.	✓ $O(bd)$ — like DFS. the current path (d d) plus $b-1$ siblings level are in memory. b=10, d=10: ≈ 100 noc
Time	$O(b^d)$ — visits each node once	$O(b^m)$ — may go very deep, but each node visited once	× $O(b^d)$ — **worse stant factor.** Nodes the root are revisite times. Overhead factor (e.g. for b=10, about 1 BFS).

Why Time Complexity Deteriorates

The key weakness of IDDFS is **repeated work**. Consider the root node A: it is expanded in *every* iteration (L=0, 1, 2, 3) — 4 times. Node B is expanded at L=2 and L=3 — twice. Only the deepest nodes (depth L) are expanded just once.

However, in a tree with branching factor b, the **number of nodes at the deepest level dominates**:

$$\text{Total expansions} = (d{+}1)\cdot 1 + d\cdot b + (d{-}1)\cdot b^2 + \cdots + 1\cdot b^d$$

$$= b^d\left(1 + \frac{1}{b} + \frac{1}{b^2} + \cdots\right) \approx b^d \cdot \frac{b}{b-1}$$

For $b = 10$, the overhead is only $\frac{10}{9} \approx 1.11\times$ — a small price for the enormous space savings ($O(bd)$ vs $O(b^d)$). This is why IDDFS is the **preferred uninformed search strategy** when the state space is large and step costs are uniform.

When IDDFS Is NOT Optimal

IDDFS finds the **shallowest** goal (fewest edges), not the lowest-cost goal. With non-uniform step costs, a deeper path with lower total cost may exist. In such cases, UCS or A* is required. For the 2024 exam graph, IDDFS found A→B→E→G (cost 16, 3 hops) while UCS found A→C→H→G (cost 13, 3 hops) — both at the same depth, but UCS found the cheaper one because it considers edge costs, not just depth.

Exam summary — IDDFS:

- **Completeness:** Inherited from BFS — guaranteed to find a solution if one exists.
- **Optimality:** Inherited from BFS — optimal for *uniform* step costs (finds shallowest solution).
- **Space:** Inherited from DFS — $O(bd)$ linear space, practical for large search spaces.
- **Time:** Worse than both BFS and DFS individually — nodes near the root are re-expanded each iteration. However, the overhead is only a constant factor ($\approx \frac{b}{b-1}$), acceptable given the space savings.

Examination 2022

Note: Search questions in 2022 are **scattered across two sections**:

- **Section A, Q.A-4(a):** DFS vs BFS theory (2 marks)
- **Section B, Q.B-4(b):** Graph trace — UCS, Greedy, A* (3 marks) — *identical search space to 2023*

Section A — Q.A-4(a): DFS vs BFS [2 marks]

Question: What is an AI Technique? Write down the advantages and disadvantages of Depth-First Search and Breadth-First Search.

AI Technique

An **AI Technique** is a method that exploits knowledge, represented in a form that:

1. **Captures generalisations** — groups situations sharing important properties together, avoiding separate representations for every individual case.
2. **Can be understood** by the people providing it — knowledge must be inspectable and modifiable.
3. **Can be easily modified** to correct errors and reflect new information.
4. **Can be used in many situations** even if not perfectly complete or accurate.
5. **Reduces the search space** by narrowing the range of possibilities that must be considered.

In short: an AI technique is a **knowledge-driven method for controlling search** so that solutions are found efficiently without exhaustive enumeration of all possibilities. DFS and BFS are two fundamental *uninformed* (blind) search techniques — they use no domain-specific knowledge, only the structure of the search tree.

Depth-First Search (DFS)

Advantages	Disadvantages
1. Low memory: Stores only the current path (b siblings per level), giving $O(bm)$ space. For deep trees, this is the decisive advantage.	**1. Not complete:** Fails on infinite-depth trees. DFS may follow an infinite branch forever and never find a solution elsewhere.
2. Simple implementation: Easy to code recursively or with a stack (LIFO).	**2. Not optimal:** The first solution found may be very deep in the tree. DFS has no mechanism to prefer shorter paths.
3. Fast to a solution if lucky: If the goal is deep and branching is low, DFS may find it with very few node expansions.	**3. Susceptible to loops:** Without a closed/visited set, DFS can cycle infinitely.
4. Backtracking-friendly: Natural fit for constraint satisfaction and puzzle-solving where partial solutions are built and abandoned.	**4. May miss shallow goals:** DFS goes deep first — a goal at depth 2 may be found only after exhausting paths to depth 50.

Breadth-First Search (BFS)

Advantages	Disadvantages
1. Complete: Guaranteed to find a solution if one exists (provided b is finite).	**1. Exponential memory:** Must store the entire frontier — $O(b^d)$ space. For b=10, d=12: 1 trillion nodes in memory. This makes BFS **infeasible** for most real problems.
2. Optimal (uniform cost): Finds the shallowest solution. If step costs are all equal, this is the optimal cost.	**2. Exponential time:** $O(b^d)$ — same as the space requirement. Practical only for shallow trees.
3. Systematic: Processes the entire tree level by level — no path is ever skipped or overlooked.	**3. Slow to find deep goals:** Must explore all nodes at depth $d-1$ before checking any at depth d. If the goal is at depth 10, all 9 shallow levels must be fully expanded first.
4. Good for shallow goals: If the goal is near the root, BFS finds it quickly.	**4. Queue implementation overhead:** Maintaining the frontier as a FIFO queue with $O(1)$ enqueue/dequeue while also tracking visited states adds complexity.

When to Use Which

Criterion	Use DFS	Use BFS
Memory is limited	**Yes** — $O(bm)$ space	No — $O(b^d)$ space
Goal is known to be deep	**Yes** — goes deep fast	No — explores all shallow levels first
Goal is known to be shallow	No — may waste time deep	**Yes** — finds it early
Need shortest path	No — not optimal	**Yes** (uniform costs)
Infinite / very deep tree	No — may not terminate	**Yes** — complete

The IDDFS resolution: Iterative Deepening DFS combines the space advantage of DFS ($O(bd)$ memory) with the completeness and optimality of BFS. For most practical uninformed search problems where memory is the bottleneck, **IDDFS is the preferred choice**. It was tested in 2024 Q2, 2023 Q2(c), 2021 Q3(b), and 2020

Q3(b) — a core exam pattern.

Section B — Q.B-4(b): Graph Trace [3 marks]

Question: Suppose you have the following search space (State, Next, Cost as given in Table 1, $h(n)$ as given in Table 2). Assume initial state A and goal state G. Show how each of the following search strategies would create a search tree to find a path from A to G: **Uniform cost, Greedy search, and A*.** At each step show which node is being expanded and the content of fringe.

The 2022 search space is identical to the 2023 search space (same states, same edges, same $h(n)$ values). The full step-by-step traces for UCS, Greedy, and A* on this graph are given in **Section 2.3** (Examination 2023, Part b(ii)). Below is the summary.

Algorithm	Path	Cost	Nodes Expanded
UCS	A → C → D → E → G	**9**	7 (A, C, D, B, E, F, G)
Greedy	A → C → F → G	15	4 (A, C, F, G)
A*	A → C → D → E → G	**9**	5 (A, C, D, E, G)

Key observation — same graph appears 3 times: The identical search space (A→B(4), A→C(1), B→D(3), B→E(8), C→D(2), C→F(6), D→C(2), D→E(4), E→G(2), F→G(8)) with the same $h(n)$ values is tested in **2020 Q2, 2022 B-4(b), and 2023 Q2**. Mastering the traces on this one graph gives you answers for 3 separate exam years. The cheapest path is always A→C→D→E→G costing 9.

Examination 2021

Note: Search questions in 2021 appear across two Q-numbers:

- **Q2 (9.75 marks):** Full search question — problem formulation + graph traces (DFID, Greedy, A*)
- **Q3(b) (1.75 marks):** DFID vs IDA* — comparative theory

Q2(a) — State Space Search Problem [0.75 mark]

Question: What is the state space search problem?

A **state space search problem** is a formal representation of a problem as a directed graph where:

- **States** are vertices — each represents a possible configuration of the world.
- **Actions** (or operators) are directed edges — transitions from one state to another.
- A **solution** is a sequence of actions (a path) from an initial state to a goal state.
- The search algorithm **systematically explores** the state space to find this path.

Formally, a state space search problem is defined by four components:

1. **Initial state** — the starting configuration.
2. **Goal test** — a function that determines whether a given state is a goal.
3. **Successor function** (operators) — given a state, returns all (action, next_state) pairs.
4. **Path cost function** — assigns a numeric cost to each path (typically additive).

The **working principle** of a basic search algorithm:

1. Initialise the fringe with the start state.
2. Loop: remove a state from the fringe; if it satisfies the goal test, return the solution path; otherwise, expand it (generate successors) and add them to the fringe.
3. Repeat until the fringe is empty (failure) or goal is found (success).

Q2(b) — Problem Formulations [3 marks]

Question: Give the initial state, goal test, successor function, cost function, and working principle of a basic search algorithm for each problem.

(i) Map Coloring Problem

Problem: Using only 4 colors, color a planar map such that no two adjacent regions share the same color.

State: A partial assignment of colors $\{R, G, B, Y\}$ to a subset of regions $\{r_1, r_2, \ldots, r_N\}$. Unassigned regions are marked as $\varnothing$. *Example:* $\{r_1{=}R,\ r_2{=}G,\ r_3{=}\varnothing, \ldots\}$.

Initial state: All N regions unassigned: $\{r_i{=}\varnothing \mid i = 1, \ldots, N\}$.

Goal test: All regions are assigned a color, and for every pair of adjacent regions (r_i, r_j), $color(r_i) \neq color(r_j)$.

Successor function: Pick an unassigned region r_i and assign it any of the 4 colors $\{R, G, B, Y\}$ that is **not** already used by any of r_i's adjacent (already-colored) regions. If no color is valid, backtrack.

Path cost: Typically 1 per assignment (uniform cost). Since all solutions assign exactly N colors, any valid solution has cost N. The goal is to find *any* valid coloring, not necessarily optimal.

Working principle: This is a **constraint satisfaction problem (CSP)**. Backtracking DFS is the natural fit: assign a color to the next variable, check constraint consistency, backtrack on conflict. DFS is preferred over BFS here because all solutions are at the same depth N, and DFS's low memory allows backtracking through a deep search tree.

(ii) Monkey & Bananas Problem

Problem: A 3-ft monkey is in a room with an 8-ft ceiling. Bananas hang from the ceiling. Two 3-ft crates are available (stackable, movable, climbable). The monkey must get the bananas.

State: A 4-tuple (*monkey_pos*, *crate1_pos*, *crate2_pos*, *has_bananas*) where positions are discrete locations in the room and *has_bananas* $\in$ {true, false}. If the monkey is on a crate, its effective height increases by the crate height (3 ft per crate; stacking 2 crates gives 6 ft height). The monkey can reach the bananas only if total height $\geq$ 8 ft.

Initial state: (*floor_position*, *initial_crate1_pos*, *initial_crate2_pos*, false) — monkey on floor, crates at their initial locations, no bananas.

Goal test: *has_bananas* = true.

Successor function (operators):

1. **Walk**(x $\rightarrow$ y): Monkey moves from position x to y on the floor (not on a crate).
2. **Push**(crate, $x \rightarrow y$): Monkey pushes a crate from x to y (must be on floor, adjacent to crate).
3. **Climb**(crate): Monkey climbs onto a crate (must be adjacent to it on the floor).
4. **ClimbDown**(): Monkey climbs down from crate to floor.
5. **Stack**(crate1, crate2): Place crate1 on top of crate2 (monkey must be on floor, crates adjacent).
6. **Grasp**(): Monkey grabs bananas (must be within reach — total height $\geq$ 8 ft).

Path cost: Number of actions (1 per action). Goal: find the **shortest** sequence to get the bananas.

Key insight: The monkey needs height $\geq$ 8 ft. One crate gives 3+3=6 ft (not enough). Two crates stacked give 3+3+3=9 ft (enough). So the solution requires: walk to crate1, push crate1 under bananas, walk to crate2, push crate2 to same spot, stack crate2 on crate1, climb, grasp.

Q2(c) — Graph Traces [5 marks]

Problem Data

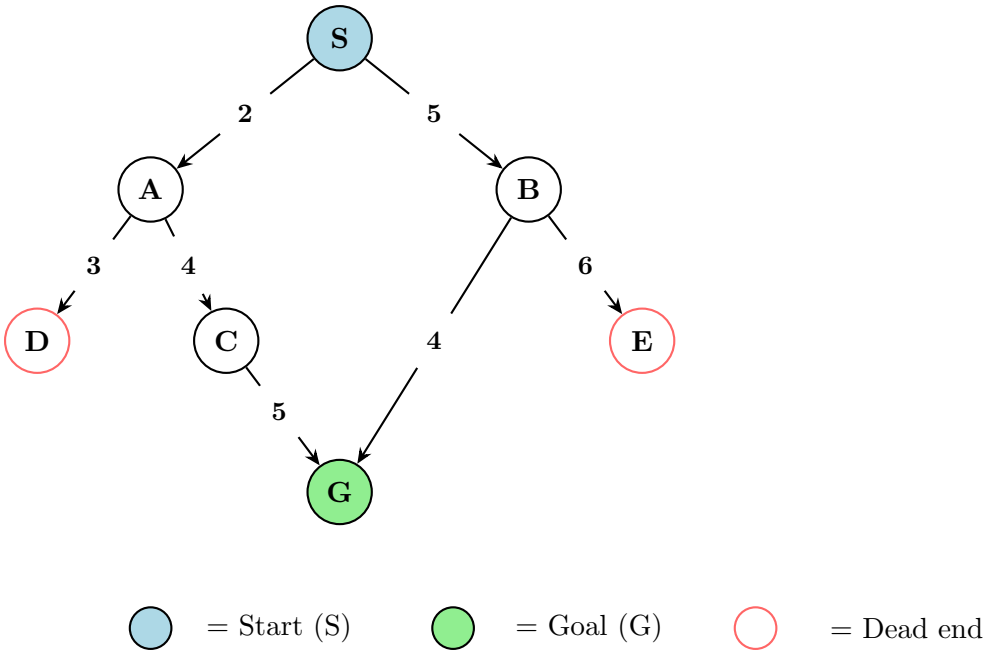
Table 1: Directed Edges and Costs

From	To	Cost
S	A	2
S	B	5
A	C	4
A	D	3
B	E	6
C	G	5
B	G	4

Table 2: Heuristic Estimates $h(n)$

Node	h(n)
S	9
A	9
B	4
C	5
D	∞
E	∞
G	0

Graph reconstruction: The exact edge costs from the 2021 past paper Figure 1 are reconstructed from the given $h(n)$ values. The heuristic is **exact** (admissible and consistent): $h(n) = h^*(n)$ for every node. D and E are dead ends (h=∞ means no path to G exists from these nodes).



Question: Assume start=S, goal=G. Show search trees for **Depth First Iterative Deepening (DFID), Greedy, and A*** — at each step show node expanded and the fringe. Also report the eventual algorithm and solution cost.

Algorithm 1 — Depth-First Iterative Deepening (DFID)

Key idea: Perform depth-limited DFS with increasing cutoff $L = 0, 1, 2, \ldots$ until goal is found. The fringe is a **stack** (LIFO). At each iteration, restart from S. Nodes are pushed in **reverse alphabetical** order so they are popped in alphabetical order.

Iteration $L = 0$:

Step	Action	Stack
1	Pop S(d=0). S $\neq$ G. At depth limit.	[ ]

No solution at $L = 0$.

Iteration $L = 1$:

Step	Action	Stack
1	Pop S(d=0). Expand: push B, then A (A on top).	A(1), B(1)
2	Pop A(d=1). A $\neq$ G. At limit.	B(1)
3	Pop B(d=1). B $\neq$ G. At limit.	[ ]

No solution at $L = 1$.

Iteration $L = 2$:

Step	Action	Stack
1	Pop S(d=0). Push B, A (A on top).	A(1), B(1)
2	Pop A(d=1). Expand (excl. S): C, D. Push D, C (C on top).	C(2), D(2), B(1)
3	Pop C(d=2). Expand (excl. A): G. Push G.	G(3), D(2), B(1)
	But C is at depth d=2, so G would be at depth 3 > L=2. Cannot push!	
	C at limit. C $\neq$ G.	D(2), B(1)
4	Pop D(d=2). D at limit.	B(1)
5	Pop B(d=1). Expand (excl. S): E, G. Push G, E (E on top).	E(2), G(2)
6	Pop E(d=2). E at limit.	G(2)
7	Pop G(d=2). G = **GOAL!**	**GOAL FOUND**

DFID Solution: $S \xrightarrow{5} B \xrightarrow{4} G$ **Depth: 2** **Cost:** $5+4=\mathbf{9}$

DFID finds the **shallowest** goal (depth 2, via S→B→G) at iteration $L=2$. DFID is complete and optimal for uniform step costs, but here with non-uniform edge costs, it finds the shallowest goal rather than necessarily the cheapest — however, in this graph, S→B→G happens to also be the optimal-cost path.

Algorithm 2 — Greedy Best-First Search

Fringe notation: Node[h] sorted ascending by h.

Step	Node	h	Fringe after expansion (sorted by h)
1	**S**	9	B[**4**], A[**9**] S→B: h=4. S→A: h=9. B has lowest h, promising.
2	**B**	4	G[**0**], E[∞], A[**9**] B→G: h=0 (goal!). B→E: h=∞ (dead end, goes to back).
3	**G**	0	**GOAL FOUND**

Greedy Solution: $S \xrightarrow{5} B \xrightarrow{4} G$ **Cost:** $5+4=\mathbf{9}$ (optimal — by luck!)

Nodes expanded: S, B, G (only 3 nodes)

Greedy found the optimal path because the heuristic h strongly prefers B (h=4) over A (h=9), and B directly connects to G. Greedy never even explored the A-branch, saving work.

Algorithm 3 — A* Search

Fringe notation: Node[g, h, f] sorted ascending by f.

Step	Node	g	h	f	Fringe after expansion (sorted by f)
1	**S**	0	9	9	B[5,4,**9**], A[2,9,**11**] S→B: f=5+4=9. S→A: f=2+9=11.
2	**B**	5	4	9	G[9,0,**9**], A[2,9,**11**], E[11,∞,∞] B→G: f=9. B→E: f=∞, deprioritised. Tie G[9],B[9]: B already expanded, G next.
3	**G**	9	0	9	**GOAL FOUND**

A* Solution: $S \xrightarrow{5} B \xrightarrow{4} G$ **Cost:** $5 + 4 = \mathbf{9}$ (optimal!)
Nodes expanded: S, B, G (only 3 nodes — same as Greedy!)
A* mirrors Greedy in this graph because the heuristic is **exact** ($h(n) = h^*(n)$ for all n), meaning $f(n) = g(n) + h^*(n)$ — the exact total cost to the goal through n. When the heuristic is exact, A* expands only nodes on optimal paths. A* never expanded A (f=11 > 9) — it was correctly identified as lying on a suboptimal path.

Algorithm Comparison for 2021 Graph:

Algorithm	Path Found	Cost	Nodes Expanded	Optimal?
DFID	S → B → G	**9**	7 (over all iterations)	Yes (this graph)
Greedy	S → B → G	**9**	3	Yes (by luck)
A*	S → B → G	**9**	3	**Yes**

Why Greedy found the optimal path here: The heuristic is **exact** — $h(n)$ equals the true optimal cost to G from every node. Since h(B)=4 (B→G costs 4) and h(A)=9 (A→C→G costs 9), Greedy correctly identifies B as the better first step. However, **this is not guaranteed in general**. With a less accurate heuristic, Greedy could be misled. The 2023 graph demonstrated exactly this: h(F)=4 looked slightly better than h(D)=5, but F→G cost 8 while D→E→G cost 6, making the D-path cheaper overall.

Q3(b) — DFID vs IDA* [1.75 marks]

Question: Differentiate between DFID and IDA* algorithms. Demonstrate how DFID incorporates the advantages of BFS and DFS with an example.

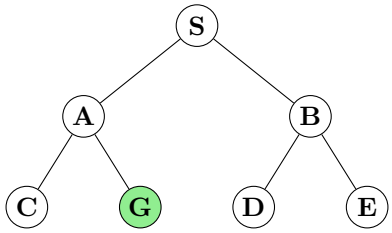
DFID (Depth-First Iterative Deepening)

DFID runs depth-limited DFS with cutoff $L = 0, 1, 2, \ldots$ until the goal is found. The cutoff is purely **structural** — it limits the number of edges (depth) from the root.

How DFID combines BFS and DFS advantages:

Property	BFS	DFS	DFID (inherits
Completeness	✓ Complete	× May loop forever	✓ **Complete** – BFS. Each iterat bounded depth; branches cannot t
Optimality	✓ Optimal (uniform cost)	× Not optimal	✓ **Optimal fo form costs** — BFS. Finds sha solution first.
Space	× $O(b^d)$	✓ $O(bm)$	✓ $O(bd)$ — fron Only stores one p

Example — Binary Tree (branching factor $b = 2$):



DFID execution on this tree:

- L=0: Visit S. Not G.
- L=1: Visit S, A, B. Not G.
- L=2: Visit S, A, C, **G** ← Found at depth 2.

BFS would: Store all nodes at depth 2 (4 nodes) in the frontier — $O(b^d)$ memory. **DFS would:** Store only the path S→A→C (3 nodes) — $O(bd)$ memory. **DFID:** Also stores only the path S→A→C (3 nodes) — $O(bd)$ memory, but with the completeness guarantee of BFS.

IDA* (Iterative Deepening A*)

IDA* is the **informed** counterpart of DFID. Instead of a depth cutoff, it uses an **f-cost cutoff**:

- Each iteration performs DFS, pruning any node where $f(n) = g(n) + h(n) > \textit{cutoff}$.
- The initial cutoff is $f(\text{start}) = h(\text{start})$.
- After each iteration, the cutoff is increased to the **smallest f-value that exceeded the previous cutoff**.
- The algorithm terminates when the goal is found (optimal) or no nodes remain below ∞ cutoff.

Key Differences

Criterion	DFID	IDA*
Cutoff type	**Depth** (structural — number of edges from root)	**f-cost** $= g(n) + h(n)$ (informed uses heuristic knowledge)
Search type	Uninformed (blind)	Informed (uses heuristic h)
Optimality guarantee	Optimal for *uniform* edge costs only	Optimal for *any* edge costs, provide h is admissible
Overhead per iteration	Lower — just checks depth $\leq L$	Higher — must compute $f(n)$ $g(n) + h(n)$ for every node visited
When to use	Unknown/unreliable heuristic, uniform costs	Good admissible heuristic availabl non-uniform costs
Space complexity	$O(bd)$ — stores current path	$O(bd)$ — stores current path (san as DFID)
Cutoff progression	$L = 0, 1, 2, \ldots$ (linear, fixed step)	$f_{cutoff} = \min\{f(n)$ n pruned in last iteration$\}$ (ada tive, data-driven)

Exam summary:

- **DFID** = IDDFS = depth-bounded iterative DFS. Uninformed. Uses depth as cutoff.
- **IDA*** = f-cost-bounded iterative DFS. Informed. Uses $f(n) = g(n) + h(n)$ as cutoff.
- Both algorithms achieve $O(bd)$ space — their key advantage over BFS and A* (which both require $O(b^d)$ space).
- DFID is to BFS what IDA* is to A*: the iterative-deepening, space-efficient version.
- In practice, IDA* is preferred for problems with good heuristics and large state spaces (e.g., the 15-puzzle).

Examination 2020

Note: 2020 has the broadest coverage of search questions — spanning Q2, Q3(b), Q3(c) in Section A and Q7(b) in Section B.

- **Q2 (8.75 marks):** Problem formulation + search space formulation + graph trace (UCS, Greedy, A*)
- **Q3(b) (2 marks):** IDDFS — how it combines BFS + DFS benefits
- **Q3(c) (1.75 marks):** BFS admissibility
- **Q7(b) (2 marks):** A* heuristic — consequences of under/overestimation

Q2(a) — Formulating a Search Problem [1.75 marks]

Question: How can you formulate a search problem? Give examples of data structures that can be used to represent a state-space both in explicit and implicit ways.

Formulating a Search Problem

A search problem is formulated by defining **four components**:

1. **Initial state** — the state in which the agent starts. *Example:* In a route-finding problem, the starting city.
2. **Goal test** — a function isGoal(s) $\rightarrow$ {true, false} that determines whether a given state s satisfies the goal condition. Can be an explicit set of goal states or an abstract condition.
3. **Successor function** Succ(s) $\rightarrow$ $\{(a, s')\}$ — given a state s, returns all (action, resulting-state) pairs reachable by legal actions.
4. **Path cost function** $g(p)$ — assigns a numeric cost to a path p (typically the sum of step costs along the path).

Together, these four components define the **search space** — a directed graph where vertices are states and edges are actions. A **solution** is a path from the initial state to any state satisfying the goal test. An **optimal solution** has the lowest path cost among all solutions.

Representing State-Spaces

1. Explicit Representation (State-Space Graph):

The entire state-space graph is stored in memory before search begins. Each state and its connections are enumerated.

Data structures used:

- **Adjacency list:** For each state, store a list of (neighbor, cost) pairs. Memory: $O(|V| + |E|)$.
- **Adjacency matrix:** A $|V| \times |V|$ matrix where entry $[i][j]$ holds the cost of the edge from state i to state j (or ∞ if no edge). Memory: $O(|V|^2)$.
- **Edge list:** Store all edges as triples (from_state, to_state, cost). Memory: $O(|E|)$.

Limitation: Only practical for **small** state spaces (e.g., the Romania map with 20 cities). For problems like the 8-puzzle ($9!/2 = 181{,}440$ states) or chess ($\sim 10^{47}$ states), explicit storage is infeasible.

2. Implicit Representation (Successor Function):

The state space is **generated on demand** during search. Only the current state and the successor function are stored; the full graph is never materialised.

Data structures used:

- **State representation:** A compact data structure encoding the current configuration. *Examples:* a tuple (x, y) for grid positions; a 2D array for the 8-puzzle board; a set of boolean variables for satisfiability problems.
- **Successor function (as code):** A function that, given a state, **computes** legal next states algorithmically — no lookup table needed. *Example:* For the Water Jug problem, successors of (x, y) are computed by the 6 pouring/filling rules applied to the current values of x and y.
- **Fringe (frontier):** A queue (FIFO for BFS), stack (LIFO for DFS), or priority queue (for UCS/A*) holding states discovered but not yet expanded. Memory depends on the algorithm, not the state-space size.
- **Closed/explored set:** A hash table or set storing states already expanded, preventing redundant work. Uses $O(|V_{\text{expanded}}|)$ memory.

Advantage: Scales to **enormous** state spaces. The 8-puzzle with 10^5 states is searched without ever storing the full graph. Chess engines use implicit representation — they generate moves on the fly and search to depth 20+ without enumerating all 10^{47} positions.

Q2(b) — Problem Formulations [2 marks]

Question: Give the initial state, goal test, successor function, and cost function for each problem.

(i) Travelling Salesman Problem (TSP)

State: $(\ell,\ V)$ where ℓ is the current city and V is the set of unvisited cities.

Initial state: (start_city, {all other $N-1$ cities}).

Goal test: $\ell =$ start_city **and** $V = \emptyset$.

Successor function: Travel$(c_i \to c_j)$ for any $c_j \in V$ (or $c_j =$ start_city if $V = \emptyset$). Removes c_j from V.

Path cost: Sum of road distances travelled. Goal: find the **minimum-cost** Hamiltonian cycle.

Complexity: $(N-1)!/2$ distinct tours — exponential. This is the canonical NP-hard problem.

(ii) Missionaries & Cannibals

State: (M_L, C_L, B) where $M_L, C_L \in \{0, 1, 2, 3\}$ are missionaries and cannibals on the left bank, and $B \in \{L, R\}$ is the boat position.

Initial state: $(3, 3, L)$.

Goal test: $(0, 0, R)$.

Successor function: Move 1 or 2 people across the river by boat. Valid combinations: $(1M, 0C)$, $(2M, 0C)$, $(0M, 1C)$, $(0M, 2C)$, $(1M, 1C)$. After each move, verify the **safety constraint**: on both banks, $M \geq C$ or $M = 0$.

Path cost: 1 per boat crossing. Goal: **minimum crossings** (known to be 11).

Q2(c) — Graph Traces [5 marks]

Question: Using the search space given (see Table), draw the state-space, then trace UCS, Greedy, and A* from A to G. At each step show which node is being expanded and the content of the fringe.

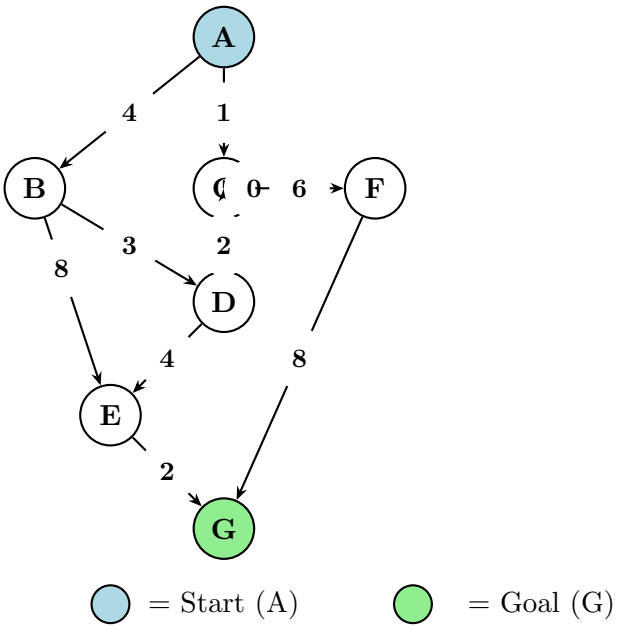
The 2020 search space is identical to the 2022 and 2023 search space (same 6 nodes A-F plus goal G, same directed edges, same $h(n)$ values). The complete step-by-step traces for all three algorithms on this graph are given in **Section 2.3** (Examination 2023, Part b(ii)). Below is the summary and the state-space diagram.

Edge Table

State	Next	Cost
A	B	4
A	C	1
B	D	3
B	E	8
C	C	0
C	D	2
C	F	6
D	C	2
D	E	4
E	G	2
F	G	8

Heuristic $h(n)$

Node	h(n)
A	8
B	8
C	6
D	5
E	1
F	4
G	0



Algorithm Results Summary:

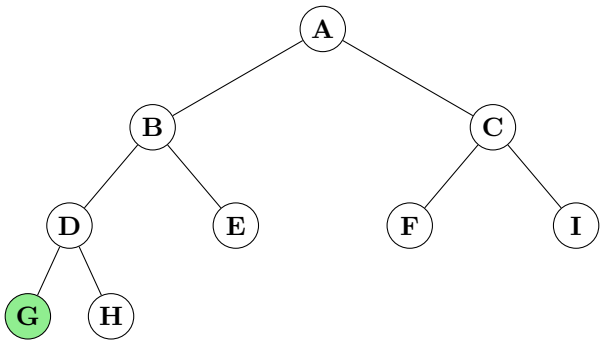
Algorithm	Path	Cost	Expansions	Optimal?
UCS	A → C → D → E → G	**9**	7	**Yes**
Greedy	A → C → F → G	15	4	No
A*	A → C → D → E → G	**9**	5	**Yes**

For the full step-by-step traces (node expanded, fringe contents at each step, discussion of why Greedy fails and A* succeeds), see **Section 2.3** (Examination 2023). The traces on this graph are among the most important in the syllabus — the same search space appears in 3 exam years (2020, 2022, 2023).

Q3(b) — IDDFS Combines BFS + DFS [2 marks]

Question: Iterative-deepening (ID) is a type of search algorithm said to combine the benefits of breadth-first and depth-first search. Explain, using a diagram or otherwise, how ID achieves these benefits.

The Problem: BFS guarantees the shortest path but needs $O(b^d)$ memory — infeasible for large search spaces. DFS uses only $O(bm)$ memory but may never terminate on infinite branches and finds suboptimal deep solutions.
The IDDFS Solution: Run depth-limited DFS repeatedly with cutoff $L = 0, 1, 2, \ldots$ until the goal is found.



L	**Nodes visited (DFS order)**	**Space used**
0	A	1 node (just A)
1	A, B, C	2 nodes (A→B, then A→C)
2	A, B, D, E, C, F, I	3 nodes (A→B→D at deepest)
3	A, B, D, **G ← found!**	3 nodes

How IDDFS inherits from each:

Property	**Inherited from BFS**	**Inherited from DFS**
Completeness	✓ BFS explores level by level, never trapped by infinite branches. IDDFS does the same — depth limits prevent infinite descent.	—
Optimality	✓ BFS finds the shallowest goal. IDDFS does the same — because it exhausts depth $d-1$ before checking depth d, the first goal found is shallowest.	—
Space	—	✓ DFS uses $O(bm)$ memory — only the current path. IDDFS replicates this: at any moment, only $O(bL)$ nodes are stored (the current path plus siblings). For b=10, L=10: ∼100 nodes vs BFS's 10 billion.
Simple implementation	—	✓ Like DFS, IDDFS uses a simple stack or recursion. The outer loop adds minimal complexity.

Cost of the compromise — Time Overhead: IDDFS re-expands nodes near the root: node A is expanded $d+1$ times, depth-1 nodes are expanded d times, etc. However, the deepest level dominates the node count. The overhead factor is $\frac{b}{b-1}$:

$$\text{Total work} = b^d\left(1 + \frac{1}{b} + \frac{1}{b^2} + \cdots\right) \approx b^d \cdot \frac{b}{b-1}$$

For $b = 10$, the overhead is only $\frac{10}{9} \approx 1.11\times$ — an acceptable price for reducing memory from $O(b^d)$ to $O(bd)$.

IDDFS = BFS completeness + BFS optimality + DFS space efficiency. The only trade-off is a modest constant-factor increase in time (≈11% for b=10), making it the **preferred uninformed search strategy** for large state spaces with uniform costs.

Q3(c) — BFS Admissibility [1.75 marks]

Question: When is breadth-first search an admissible search strategy? Briefly explain.

What "Admissible" Means

In search, an algorithm is **admissible** if it **always finds the optimal (lowest-cost) solution** when one exists. The term is most commonly applied to heuristics ($h(n) \leq h^*(n)$), but it can also describe complete search algorithms.

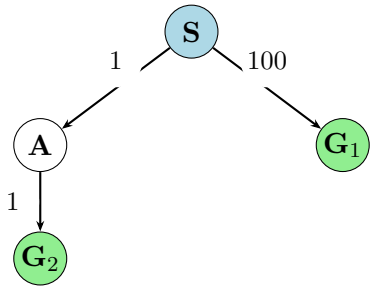
When BFS is Admissible

BFS is admissible **when all step costs are equal (uniform)**. Under this condition:

1. BFS explores the state space **level by level**: all nodes at depth 0, then depth 1, then depth 2, ...
2. Since every step costs the same amount (say, 1), the **depth of a node equals its path cost**: $g(n) = \text{depth}(n)$.
3. Therefore, the **first goal node encountered** is at the shallowest depth, which corresponds to the **minimum path cost**.
4. BFS returns this shallowest goal — which is optimal.

When BFS is NOT Admissible

BFS is **not admissible when step costs are non-uniform**. Consider:



BFS would find G_1 first (depth 1, cost 100) and return it as the solution. But the optimal path is $S \to A \to G_2$ (depth 2, cost $1+1=2$). BFS returns the **shallowest** goal, not the **cheapest**. The solution is suboptimal, so BFS is not admissible when edge costs differ.

Formal Statement

BFS is admissible iff all step costs are equal.

When costs are uniform, depth $\propto$ cost, and the shallowest goal is the cheapest. When costs vary, BFS returns the shallowest goal regardless of cost — this may be suboptimal. For non-uniform costs, use **UCS** (Uniform-Cost Search) instead — it expands by $g(n)$ rather than depth, guaranteeing optimality for any non-negative edge costs.

Q7(b) — A* Heuristic: Underestimate vs Overestimate [2 marks]

Question: What will happen if h' underestimates h in the A* algorithm? What will happen if h' overestimates h?

Notation clarification: In standard A* notation, $h(n)$ is the heuristic estimate of the cost from n to the goal, and $h^*(n)$ is the *true* optimal cost. The question uses h' for the heuristic and h for the true cost. We adopt the standard convention below.

Case 1: Heuristic UNDERESTIMATES ($h(n) \leq h^*(n)$ — Admissible)

When the heuristic never overestimates the true cost to the goal (it is **admissible**):

Consequence	Explanation
A* is optimal — it will find the lowest-cost solution.	The proof: A* expands nodes in order of $f(n) = g(n) + h(n)$. Since $h(n) \leq h^*(n)$, we have $f(n) \leq g(n) + h^*(n) =$ true total cost through n. The first time A* expands a goal node G, $f(G) = g(G) + 0 = g(G)$. For any unexpanded node n on a cheaper path to a different goal, $f(n) \leq$ true cost of that cheaper path $< g(G)$. So A* would have expanded n before G — contradiction. Therefore the first expanded goal must be optimal.
May expand more nodes than with a perfect heuristic.	A severe underestimate (e.g., $h(n) = 0$ for all n, which reduces A* to UCS) leads to more node expansions. The closer h is to h^*, the fewer nodes A* expands.
Still complete — will find a solution if one exists.	Admissibility does not affect completeness. A* with any heuristic eventually explores the entire (finite) search space if needed.

Example — admissible (underestimate): $h(\text{Arad}) = 366 \leq h^*(\text{Arad}) = 418$. A* finds the optimal path Arad $\rightarrow$ Sibiu $\rightarrow$ Rimnicu $\rightarrow$ Pitesti $\rightarrow$ Bucharest (cost 418).

Case 2: Heuristic OVERESTIMATES ($h(n) > h^*(n)$ for some n)

When the heuristic **overestimates** the true cost for at least one node (it is **not admissible**):

Consequence	Explanation
A* may NOT be optimal — it can return a suboptimal solution.	If $h(n)$ overestimates the remaining cost through node n, then $f(n)$ is inflated. A* may postpone expanding n in favor of nodes whose f-values are lower but lie on suboptimal paths. A suboptimal goal may be expanded and returned before the optimal path is discovered.
The optimal path may be "hidden."	A* considers nodes in order of f-value. An overestimating heuristic makes good nodes look worse than they are (artificially high f), so they sit deeper in the fringe while A* explores suboptimal-looking nodes with deceptively low f.
Still complete — overestimation does not affect completeness.	A* will still find *a* solution (the search is still systematic). It just may not be the *best* one.

Example — inadmissible (overestimate): Suppose $h(\text{Fagaras}) = 200$, but the true cost from Fagaras to Bucharest is $h^*(\text{Fagaras}) = 99$. Then $f(\text{Fagaras}) = 239 + 200 = 439$, while the path through Pitesti has $f(\text{Pitesti}) = 317 + 101 = 418$. The Fagaras path looks worse and is explored later. If Sibiu→Fagaras→Bucharest (cost $239 + 99 = 338$) were actually optimal but h made it look worse, A* might return the Pitesti route (cost 418) — suboptimal.

Summary

Property	h **underestimates (admissible)**	h **overestimates (inadmissible)**
Optimality	**Guaranteed** ✓	**Not guaranteed** ×
Node expansions	More than with exact h	Unpredictable — may expand fewer nodes
Completeness	Guaranteed ✓	Guaranteed ✓
Example	$h(n) = 0$ (UCS — slow but optimal)	$h(n) > h^*(n)$ (fast but potentially wrong)
Design rule	**Always make h admissible**	**Never intentionally overestimate**

Exam key point: The **admissibility condition** $h(n) \leq h^*(n)$ for all n is the **single most important property** of a heuristic in A*. It is the necessary and sufficient condition for A* optimality (on tree-search; on graph-search, consistency/monotonicity is also required for optimality without reopening nodes). An admissible heuristic guarantees you won't miss the best path; a well-chosen admissible heuristic (close to h^*) makes the search efficient.

Hill Climbing & Simulated Annealing (Topic 9) — All Years

Chapter 4 (slides 61–75) covers local search. Questions appear in **2021 Q3a, 2022 B-4a, 2023 Q3a, and 2024 Q4b**. Core concepts tested: Steepest-Ascent HC algorithm, three failure modes + remedies, HC (deterministic) vs SA (non-deterministic), and SA's escape mechanism via probabilistic acceptance.

Theory — Hill Climbing

What is Hill Climbing?
Hill Climbing (gradient ascent/descent) is a **local search** algorithm. It starts with an initial state and iteratively replaces the current state with its best successor. It keeps only **one state in memory** (OPEN = {current node}) and uses only the heuristic evaluation function $h(n)$ — no path cost, no backtracking.

Simple Hill Climbing (from slide 68):

1. Evaluate initial state. If goal, return it; else make it current.
2. Loop: (a) select an operator not yet applied and apply it to get a new state; (b) if new state is goal → return it; if better than current → make it current; else continue loop.

Steepest-Ascent Hill Climbing (slide 69) — the version the exam asks for:
1. Set *current* := initial state.
2. Loop:
(a) Generate **all** successors of *current*.
(b) Let *best* := successor with the **lowest** $h(n)$ (i.e. best evaluation).
(c) If $h(best) < h(current)$: set *current* := *best*; continue loop.
(d) Else: no improvement found. **Return** *current* (local optimum).

Simple HC vs Steepest-Ascent HC:

Simple HC	Steepest-Ascent HC
Moves to the **first** successor that is better. Stops testing once any improvement found.	Evaluates **all** successors. Moves to the **best** one.
Faster per step.	Follows the true gradient; less prone to premature stopping.

Three Drawbacks of Hill Climbing (Slide 65 — Memorise This)

The exam asks this directly and repeatedly: "Describe the problems HC may reach. How can each be dealt with?"

1. Local Maximum

Definition (slide 65): A state that is **better than all its neighbours** but is **not the global optimum**.

What happens: HC halts at this peak because every adjacent state is worse. The algorithm incorrectly returns the local peak as the solution.

Example: A drone route better than all routes reachable by a single waypoint change, yet many superior routes exist requiring 2+ simultaneous changes.

Remedies:

1. **Backtrack** to a previously visited state and try a different direction.
2. **Random restart** — restart from a different (randomly chosen) initial state.
3. Use **Simulated Annealing** — probabilistically accept worse states to escape.

2. Plateau

Definition (slide 65): A **flat area** of the search space where a set of neighbouring states all have the **same evaluation value**.

What happens: HC cannot determine which direction to move. It may wander aimlessly or halt prematurely.

Example: Multiple drone routes of identical total flight cost; HC has no information to guide its next modification.

Remedies:

1. **Allow sideways moves** (accept equal-value successors) up to a maximum count before giving up.
2. **Random restart.**

3. Ridge

Definition (slide 65): A **special kind of local maximum** — a region higher than surrounding areas that **itself has a slope** (which one would like to climb). The trouble is that the available operators cannot move along the ridge; each move off the ridge is downhill.

What happens: HC reaches the foot of the ridge and cannot climb it because every available operator moves the agent sideways off the ridge and downhill.

Example: A drone route corridor where the global optimum runs along a narrow geographical path — any single waypoint change moves the drone off the corridor and worsens the route.

Remedies:

1. **Apply two operators simultaneously** to move diagonally along the ridge.
2. **Change the heuristic** — use a **global heuristic** (Knight's Book: reward correct support structure) rather than a local one that cannot see the ridge.
3. **Random restart.**

HC problems cheat-sheet (3 lines):
Local Max: Better than all neighbours but not globally best. Remedy: backtrack or random restart.
Plateau: All neighbours equal — no direction to move. Remedy: allow sideways moves or restart.
Ridge: Sloped region higher than surroundings — operators can't traverse it. Remedy: apply 2 operators simultaneously or change heuristic.

HC vs BFS Comparison (Slide 70)

Criterion	Hill Climbing	Best-First Search
Memory	**O(1)** — only current state. All other successors are discarded and **never reconsidered**.	$O(b^d)$ — entire frontier stored. Rejected nodes kept and can be revisited.
Backtracking	**None.** Once a move is made, discarded states are gone forever.	**Full.** If selected path fails, BFS returns to the next-best frontier node.
Stuck behaviour	**Halts permanently** when no better successor exists.	**Continues** — switches to next frontier node even if its value is lower than the just-explored state.
Determinism	**Deterministic** — same initial state always produces same path and same local optimum. Zero randomness.	**Deterministic** — consistent tie-breaking makes A*/Greedy deterministic too.
Completeness	**Not complete** — terminates at local optima, plateaux, ridges.	A*: **complete**. Greedy: incomplete on infinite graphs.

Simulated Annealing — Escape Mechanism

Motivation: HC always gets stuck at the nearest local optimum. SA escapes by **occasionally accepting worse states**, inspired by the physical process of slowly cooling metal into a minimum-energy crystal.

SA core idea (slide 71): Maintain a temperature T (starts high, decreases to 0).
At each step: randomly select a successor. Compute $\Delta E = \text{VALUE[next]} - \text{VALUE[current]}$.
If $\Delta E > 0$ (improvement): **always accept.**
If $\Delta E \leq 0$ (worse): accept with probability $e^{\Delta E/T}$.
As $T \to 0$: worse moves become impossible. SA converges to hill climbing.

SA Algorithm (slides 72–73):

current ← INITIAL-STATE; $T := T_0 \gg 0$
forever do:
if $T \leq T_{\text{end}}$ **then return** *current*
next ← randomly generated new state
$\Delta E = f(next) - f(current)$
current := *next* with probability $p_{\Delta E} = \dfrac{1}{1 + e^{\Delta E/T}}$
Cooling schedules: $T := T \times \alpha$ (geometric, $0 < \alpha < 1$) or $T_k := T_0 / \log(1 + k)$

How SA addresses each HC failure mode:

HC failure	HC behaviour	SA behaviour
Local maximum	Halts permanently	Accepts a worse neighbour with probability $e^{\Delta E/T}$ — escapes the local peak
Plateau (ΔE=0)	Wanders or halts	$e^{0/T} = 1 \rightarrow$ always accepts lateral moves; traverses the plateau
Ridge	Cannot traverse	Temporarily accepts the downhill move off the ridge; can reach the other side

SA properties (slide 74): SA is **complete and optimal** given a **slow enough cooling schedule** (in probability). With fast cooling: fast but suboptimal. With slow cooling: near-optimal.
SA is non-deterministic — random successor selection + probabilistic acceptance → different runs may produce different solutions.

2021 — Q3(a): Steepest-Ascent HC + Problems [4 marks]

Question: Describe the Steepest-Ascent Hill Climbing algorithm. What problems can HC reach? Discuss how to deal with each.

Algorithm: (see Section 6.1 pseudocode above — generates all successors, selects best, halts if no improvement).
Three problems + remedies:

1. **Local Maximum.** Better than all neighbours but not globally optimal. HC halts, reporting a suboptimal answer.
   Remedy: Restart from a different initial state; backtrack and try a different branch.

2. **Plateau.** All neighbouring states have equal evaluation value. HC has no direction to move.
   Remedy: Allow sideways moves up to a fixed limit; random restart.

3. **Ridge.** Region higher than surroundings with a slope; operators cannot traverse it directly.
   Remedy: Apply two operators simultaneously; use a global heuristic.

2022 — B-4(a): HC Algorithm + Problems [2 marks]

Question: Write the Steepest-Ascent HC algorithm. What problems does HC reach? How can they be dealt with?

Algorithm: At each step, evaluate ALL successors. Move to the one with the best $h(n)$. Halt if no successor is better.

Problem	Definition	Remedy
Local Maximum	Better than all neighbours; not globally optimal	Random restart; backtrack
Plateau	All neighbours equal; no direction	Allow sideways moves; restart
Ridge	Higher than surroundings but has a slope operators can't follow	Apply 2 operators simultaneously; change heuristic

2023 — Q3(a): Deterministic/Non-deterministic + HC + SA [4 marks]

Question: Differentiate between deterministic and non-deterministic search. Discuss HC limitations for drone route planning. How does SA address them?

Deterministic vs Non-Deterministic

Criterion	Deterministic	Non-Deterministic
Definition	Same initial state + same inputs → always the **same sequence of states** and same result.	Incorporates **randomness** → different runs produce different paths and possibly different results.
Examples	HC (Steepest-Ascent), A*, UCS, Greedy BFS	Simulated Annealing, Genetic Algorithms
Advantage	Reproducible; easy to analyse.	Escapes local optima; explores more of the space.
Disadvantage	Always stuck at the same local optimum for the same input.	Non-reproducible; harder to guarantee a specific solution.

HC Limitations for Drone Route Planning

State = current route; evaluation = total flight cost; operators = change one waypoint.

1. **Local Maximum.** HC finds a route better than all single-waypoint modifications. Globally better routes requiring multi-waypoint changes are unreachable. HC terminates prematurely.
2. **Plateau.** Multiple drone routes have identical total cost. HC cannot identify a direction to improve and stalls.
3. **Ridge.** The optimal route runs along a narrow cost corridor. Any single waypoint change pushes the drone off the corridor and increases cost. HC cannot traverse the ridge.

How SA Addresses Each Limitation

1. **Local Maximum:** SA accepts a worse route with probability $e^{\Delta E/T}$, escaping the trap and exploring routes that HC would never visit.
2. **Plateau:** Since $\Delta E = 0$, the acceptance probability $e^{0/T} = 1$. SA freely traverses the plateau.
3. **Ridge:** SA temporarily accepts the off-ridge (downhill) move. From there, it may climb the other side of the ridge to reach the global optimum.

Key exam distinction (2023): HC is **deterministic** — same input → same stuck point. SA is **non-deterministic** — randomness enables escape. SA's temperature T: high T = broad exploration; low T = fine-grained refinement near optimum.

2024 — Q4(b): HC det/non-det + 3 Limitations + SA [4 marks]

Question: Is HC deterministic or non-deterministic? Identify 3 limitations of HC for autonomous drone route planning. Explain how SA addresses these limitations.

HC is Deterministic

Hill Climbing is deterministic. Given the same initial state and evaluation function, Steepest-Ascent HC always generates the same successors, always selects the same best one, and always terminates at the same local optimum. There is no random element.

SA is non-deterministic — it randomly selects the next state and probabilistically decides whether to accept it with probability $e^{\Delta E/T}$.

3 Limitations for Drone Route Planning + SA Solutions

1. **Local Maximum.**
   HC finds a route better than all single-modification alternatives but misses globally superior routes needing multi-point changes. Drone follows a suboptimal path.
   SA: Accepts worse routes with probability $e^{\Delta E/T}$, escaping the local optimum to find better routes.

2. **Plateau.**
   Multiple drone routes have equal total cost. HC has no gradient to follow and stalls indefinitely.
   SA: $e^{0/T} = 1 \rightarrow$ SA always accepts equal-cost moves; crosses the plateau freely.

3. **Ridge.**
   Optimal route runs along a narrow corridor. Every individual waypoint change moves the drone off-corridor and worsens cost. HC cannot traverse the ridge.
   SA: Temporarily accepts the off-corridor worse route. From that position SA may locate and ascend the other slope, reaching the globally optimal corridor.

2024 summary:
HC = **deterministic**. SA = **non-deterministic** (probabilistic acceptance $e^{\Delta E/T}$).
HC's 3 drone failures: **local maximum** (suboptimal route), **plateau** (equal-cost routes, no direction), **ridge** (narrow corridor unreachable by single changes).
SA temperature schedule: T starts high (broad exploration) $\rightarrow$ decreases to 0 (no worse moves accepted) $\rightarrow$ converges to optimum.

Constraint Satisfaction Problems (Topic 10) — All Years

Chapter 5 (slides 1–23, R&N 5.1–5.3, 4.3) covers CSPs. Questions appear in **2021 A-Q3c, 2022 B-4d, 2023 A-Q3c, and 2024 A-Q4 (part)** — always as a 1–3 mark sub-part bundled with HC/SA/Fuzzy, never standalone. Two recurring patterns: (a) **formulate** a problem as a CSP (variables, domains, constraints), and (b) **trace** the cryptarithmetic SEND+MORE=MONEY puzzle.

Theory — What Is a CSP?

A **Constraint Satisfaction Problem (CSP)** is defined by a set of **variables** X_i, each with a **domain** D_i of possible values, and a set of **constraints** C. The aim is to find an assignment of values to all variables, drawn from their domains, such that **none of the constraints are violated**.

Varieties of constraints (slide 12):

- **Unary** — involves a single variable, e.g. $M \neq 0$.
- **Binary** — involves a pair of variables, e.g. $SA \neq WA$.
- **Higher-order** — involves 3+ variables, e.g. $Y = D + E$ or $Y = D + E - 10$.
- **Inequality (continuous)** — e.g. $EndJob1 + 5 \leq StartJob3$.
- **Soft (preferences)** — e.g. "an 11am lecture is preferable to an 8am lecture".

Path Search vs. CSP — the key distinction (slide 16): In a path-search problem (e.g. Rubik's Cube), the goal state is easy to recognise but hard to reach. In a CSP (e.g. n-Queens, map-coloring), the **difficult part is knowing the goal state itself** — once a complete, consistent assignment is found, "how we got there" is irrelevant.

Backtracking Search — The Standard Algorithm

Standard search formulation (slide 12): states = partial assignments; initial state = the empty assignment; successor function = assign a legal value to one unassigned variable (fail if none exists — *not fixable*); goal test = assignment is complete. Every solution appears at **depth** n (for n variables) ⇒ **depth-first search** is the natural fit.

Backtracking search (slides 13–14):

Since variable assignments are **commutative** ([WA=*red* then NT=*green*] is the same state as [NT=*green* then WA=*red*]), we only need to consider assigning **one variable at a time** at each search-tree node. Depth-first search restricted to single-variable assignments **is** backtracking search:

1. Pick an unassigned variable; try a value from its domain consistent with current assignments.
2. Recurse on the remaining unassigned variables.
3. If no value is consistent, **backtrack** — undo the last assignment and try the next alternative.

Backtracking is the **basic algorithm for CSPs** — it can solve n-Queens for $n \approx 25$.

Improvements to Backtracking (slides 17–23)

Three general-purpose questions drive the speed-ups: *(1) which variable to assign next? (2) which value to try first? (3) can failure be detected early?*

Heuristic	Idea
Minimum Remaining Values (MRV)	Choose the **variable** with the **fewest legal values** left — the most likely to fail soonest, so fail fast and prune the tree early.
Degree heuristic	Tie-breaker for MRV: choose the variable involved in the **most constraints** on remaining unassigned variables.
Least Constraining Value (LCV)	Given a variable, choose the **value** that rules out the **fewest** options for neighbouring variables — keeps the most flexibility for the rest of the search.
Forward checking	After assigning a variable, **prune** that value from the domains of its neighbours; **terminate immediately** if any variable's domain becomes empty (failure detected before it is reached).

CSP cheat-sheet: CSP = ⟨Variables, Domains, Constraints⟩. Solved by **backtracking** (DFS + single-variable assignment + undo on failure). Sped up by **MRV** (fewest legal values), **degree heuristic** (most constraints, tie-break), **LCV** (least re-

strictive value), and **forward checking** (prune neighbour domains, fail early). Combining these heuristics makes problems like 1000-Queens tractable.

2021 — A-Q3(c) & 2023 — A-Q3(c): Formulate Map Coloring as a CSP [part marks]

Question (recurring, 2021 & 2023): Formulate the map-coloring problem as a Constraint Satisfaction Problem — identify the variables, domains, and constraints.

Problem: Colour each region of a map so that no two adjacent regions share the same colour, using a fixed palette (e.g. $\{red, green, blue\}$).

Variables: One per region: WA, NT, Q, NSW, V, SA, T.

Domains: $D_i = \{red, green, blue\}$ for every region i.

Constraints (binary): Adjacent regions must receive different colours: $WA \neq NT$, $WA \neq SA$, $NT \neq SA$, $NT \neq Q$, $SA \neq Q$, $SA \neq NSW$, $SA \neq V$, $Q \neq NSW$, $NSW \neq V$. (No constraint on T — Tasmania is an island, adjacent to nothing.)

Example solution (a complete, consistent assignment):

$$\{WA{=}red,\ NT{=}green,\ Q{=}red,\ NSW{=}green,\ V{=}red,\ SA{=}blue,\ T{=}\text{any colour}\}$$

One-line definition to open the answer: "A CSP is defined by a set of variables, each with a domain of possible values, and a set of constraints; a solution is an assignment of values to all variables that satisfies every constraint." Then plug in the three components above — that alone earns full marks on a 1–3 mark formulation sub-part.

2022 — B-4(d): Trace the Cryptarithmetic SEND + MORE = MONEY CSP [3 marks]

Question: What is a CSP? Trace how the cryptarithmetic puzzle SEND + MORE = MONEY is solved as a CSP.

Setup (slide 7):

Variables: S, E, N, D, M, O, R, Y (each letter is a digit).

Domains: $D_i = \{0, 1, 2, \ldots, 9\}$.

Constraints:

- **Unary:** $M \neq 0$ and $S \neq 0$ (no leading zeros).
- **All-different:** every letter maps to a distinct digit.
- **Higher-order (column arithmetic with carries X_1, X_2, X_3):** $D + E = Y + 10X_1$; $N + R + X_1 = E + 10X_2$; $E + O + X_2 = N + 10X_3$; $S + M + X_3 = O + 10M$.

Trace (column-by-column, right to left, backtracking on conflict):

1. **Units column:** assign D, E such that $D + E = Y$ or $Y + 10$ (carry X_1=1 if ≥ 10). Backtrack if the resulting Y clashes with an already-used digit.
2. **Tens column:** assign N, R such that $N + R + X_1 = E$ or $E + 10$ (carry X_2). Any clash with previously assigned digits forces backtracking to the units column to try a new (D, E) pair.
3. **Hundreds column:** assign O such that $E + O + X_2 = N$ or $N + 10$ (carry X_3).
4. **Thousands column:** $S + M + X_3 = O + 10M$. Since the result MONEY has one more digit than the addends, the final carry X_3 **must propagate** to give the leading digit, forcing $M = X_3 = 1$ — and consequently $S = 8$ or 9.
5. **Consistency check:** verify all eight letters are pairwise distinct (the all-different constraint). If any clash remains, backtrack to the most recent choice point and try the next legal value.

Unique solution found:

$$\{S{=}9,\ E{=}5,\ N{=}6,\ D{=}7,\ M{=}1,\ O{=}0,\ R{=}8,\ Y{=}2\} \ \Rightarrow\ 9567 + 1085 = 10652$$

Why this is a CSP, in one line: the puzzle has variables (letters), domains (digits 0–9), and constraints (no leading zeros, all-different, column-sum-with-carry); backtracking search assigns digits column by column and undoes a choice the moment any constraint is violated — exactly the general CSP solution method.

2024 — A-Q4 (part): CSP Short Note (combined with HC/SA/Fuzzy) [part marks]

Question: Short note on Constraint Satisfaction Problems (asked as one part of a combined HC/SA/Fuzzy/CSP question).

Short-note answer: A Constraint Satisfaction Problem is defined by a set of **variables**, each with a **domain** of possible values, and a set of **constraints** restricting which combinations of values are legal; a solution is a complete assignment that violates none of the constraints. Classic examples are **map-colouring** (variables = regions, domains = colours, constraints = adjacent regions differ) and **cryptarithmetic** (variables = letters, domains = digits 0–9, constraints = no leading zeros, all-different, column sums with carries). CSPs are solved by **backtracking search** — depth-first search that assigns one variable at a time and undoes a choice as soon as a constraint is violated — and sped up with **MRV**, **degree heuristic**, **least-constraining-value**, and **forward checking**. Real-world CSPs include timetabling, scheduling, and hardware/transport configuration (slide 11).

If only a one-liner is asked: "A CSP is a problem defined by variables with domains and a set of constraints; solving it means finding an assignment satisfying every constraint, typically via backtracking search (e.g. map-colouring, n-Queens, cryptarithmetic)."